

Assignment A: Inference Serving Engine

一个请求在系统里的一生

回顾式讲义 · 基于 llm-serving Week 1-3

Version 0.1.0 · 2026-07-29

1 讲义总览

这份讲义回顾 llm-serving 这个仓库的前三周：一个从零写起的 LLM inference serving engine。

它不是作业。代码已经写完了，题目已经做过了。讲义要做的是把当时的推理过程重新摊开——特别是那些被否决的方案，因为在这个仓库里，被否决的方案比留下来的代码更值钱。

1.1 做成了什么，没做成什么

CS336 的 handout 在这个位置写 “What you will implement”。回顾式讲义没有将来时，所以这里必须同时写另一半：到此为止。

已经建成的：

1. 请求的数据契约与生命周期状态 (RequestData / RequestStatus / TokenizedData)
2. tokenizer 的隔离缝——一个约 30 行的包装，换掉底层实现时只有它要改
3. 单请求的 prefill → decode 循环，手写驱动模型而不是调 generate()
4. 协作式 async 调度 (ADR-001 称之为 “Weak Async”)，一个 consumer 循环
5. 端到端差分测试：输出与 HuggingFace 的 generate(do_sample=False) token 级完全一致

没有建成的，讲义也不会假装讲了：

1. continuous batching——§4 讲的是它的设计，Week 3 至今没有代码
2. scheduler 与 admission policy
3. 真正的 KV cache manager（现在是每请求一份，请求结束就扔）
4. output/ 层：detokenize 与流式回传，目录是空的
5. HTTP 层。RequestReceiver.submit_request() 就是这个系统对外的 API
6. CI、lint 配置、benchmark

1.2 你可以亲手跑什么

三条，都不需要 GPU。

```
uv sync
uv run pytest -q           # 44 passed, 1 skipped
uv run python draft.py     # sleep(0) 赛跑实验，见 §3
```

Resource requirements: CPU 或 MPS 即可。第一次跑 pytest 会下载 gpt2 权重（约 500 MB），之后约 70 秒跑完。被 skip 的那一个是跨仓库集成测试，需要一份训练好的 checkpoint，见姊妹篇。

1.3 仓库长什么样

```
inferenceLM/
  data/           三个数据契约           -> §2
  request_receiver/ 接收与 tokenize       -> §2
  engine/
    inference_engine.py 编排层           -> §3
    lm_engine.py         纯计算层         -> §3
    transformer_lm_adapter.py 见下方说明
  output/         【空】detokenize/流式回传，尚未开始
  benchmarks/     【空】
  docs/
```

```

designs/DESIGN.md          v0 架构设计（部分已被取代，见 §1.5）
decisions/adr00{0,1}~*.md 两份 ADR
progress/, retro/         周计划与复盘
draft.py                  独立实验脚本，不属于包    -> §3
main.py                   打印一行问候的桩，无人调用
LICENSE                   【0 字节】
README.md                 【仍是脚手架】写着 "[Project Name — TO BE DECIDED]"

```

两个空目录不是遗漏，它们是这个项目进度的一部分，所以要列出来。

`inferenceLM/engine/transformer_lm_adapter.py` 需要单独说一句：它在物理上属于这个仓库，但它要讲的事情属于姊妹篇——它是把另一个项目里自己训练的模型接进 `LMEngine` 的适配层。这份讲义不讲它，见 Assignment B 第 4 章。

规模上：`inferenceLM/` 下的 Python 约 514 行，`tests/` 约 880 行，`docs/` 下的 markdown 约 1502 行。文档是包代码的三倍——对一个 portfolio 项目来说这个比例本身就是一句论点。

1.4 怎么用这份讲义

橙色的 **Checkpoint** 框是自测题，不是作业。用法：先合上代码，把题干答完，再看下半格的参考答案和代码位置。

三条约定：

- 代码是唯一的事实来源。每个 Checkpoint 都给了 `file.py:line`。参考答案和代码冲突时以代码为准，并且请把讲义改掉。
- 灰框和深红框里的内容是错的，故意保留。灰色的「常见误解」装的是当时真的这么想过、后来被推翻的说法；深红色的「撤回」装的是曾经报出去、后来撤回的结论。删掉它们会让讲义看起来更聪明，但会丢掉最有用的部分。
- 建议读两遍。第一遍顺读，Checkpoint 全跳过；第二遍只做 Checkpoint，答不上来再翻回去。想要一份不带答案的版本：

```

latexmk -xelatex -jobname=assignment-a-quiz \
  -pretex='\def\HANDOUTQUIZ{ }' -usepretex assignment-a.tex

```

1.5 文档时间线，以及哪一份现在算数

这是全篇最该先看的一张表。这个仓库的文档不是同时写的，也不是彼此一致的——照着过期的那份建，会建错东西。

两处值得单独记住：

`adr001` 的 Status 是 Proposed。ADR 模板里 Proposed 的意思是「提出，待定」，但代码已经完全照它建好了。这是流程与现实脱节的一个具体例子——§2 里那两个从未被赋值的枚举值是同一件事在类型系统里留下的痕迹。

`week3-discussion.md` 取代了 `DESIGN.md`，但它自己内部也有一处不一致。这一点留到 §4 展开。

文档	日期	Status	现在的效力
designs/DESIGN.md	2026-04-09	Approved	写于 Week 3 re-scope 之前。容量规划那部分仍然有效；静态预分配 KV cache 与 2:1 prefill 优先调度已被取代
decisions/adr000-request.md	2026-04-11	Accepted	现行
progress/week1-tasks.md	—	—	已完成
retro/week1-retro.md	—	—	三条失败模式与对应动作
progress/week2-tasks.md	—	—	已完成
decisions/adr001-inference.md	2026-04-15	Proposed	代码全部照它实现了,但这份 ADR 至今没有升到 Accepted
retro/week2-retro.md	—	—	853 行, 全仓库最长; §3 的主要素材
progress/week3-discussion.md	2026-04-21	—	batching 与 scheduling 上的现行 SSOT, 取代 DESIGN.md
progress/future-tasks.md	—	—	DESIGN.md 时代的产物, 与 week3 的方案不完全一致

Table 1: 文档时间线与现行效力。没有 Date / Status 字段的文档如实留空。

2 请求的形状：数据模型与系统边界

一个 serving 系统的第一个真正的设计决定，不是“用哪个模型”，也不是“用不用 continuous batching”。第一个决定是：request 在这个系统里到底是什么。

这问题看起来随手就能答——建一个类，把用户给的东西都装进去，需要什么再加字段。代价要过几周才显形：全塞进一个类，engine 层就会开始知道 user_id 和 timestamp；等到要 batch 的时候，你在 batch 一堆和 forward pass 毫无关系的字段，而真正要拼在一起的那点东西（token 列表和它的长度）被埋在中间。到那时再拆，拆的不是一个类，是所有已经写好的调用点。

Week 2 在这里选了两个类，不是一个。这一节看它买到了什么、代价是什么、以及它在类型系统里留下的疤。涉及的代码在 inferenceLM/data/ 和 inferenceLM/request_receiver/ 下面，不到一百行；分量不在行数上，在于它决定了后面每一层能看见什么。

2.1 两个类而不是一个

RequestData 是一个 pydantic BaseModel：

```

21     user_id: str
22     request_id: str
23     timestamp: float
24     status: RequestStatus = RequestStatus.PENDING
25     prompt_text: str
26     max_token_length: int = 20 # prompt + response <= max_token_length
27     do_sample: bool = False
28     generated_tokens: List[int] = Field(default_factory=list) # 提前准备buffer

```

TokenizedData 是裸的 Python class，连 @dataclass 都没用：

```

12     def __init__(self, request_id: str, tokens: List[int]):
13         self.request_id = request_id
14         self.tokens = tokens # the tokenized input data for the request
15
16     def __str__(self):
17         return f"Tokenized request: {self.request_id}"
18
19     def __len__(self):
20         return len(self.tokens)

```

差别不只是“一个大一个小”。分界线是谁进 queue：submit_request() 把 RequestData 存进 request_store（inferenceLM/request_receiver/request_receiver.py:52），往 pending_queue 里放的却是另一个东西：

```

42     # 在接收到 user 的 input text 之后，在function内部重新包装成一个 RequestData
43     class
44         request_data = RequestData(
45             prompt_text=prompt_text,
46             user_id=user_id,
47             request_id=str(RequestReceiver.index),
48             timestamp=time(),
49             status=RequestStatus.PENDING,
50             max_token_length=max_length,
51             do_sample=do_sample
52         )
53         self.request_store[request_data.request_id] = request_data # 存 request_id :
54         request_data
55         RequestReceiver.index += 1
56
57     # Create tokenized data
58     tokenized_prompt = self.tokenizer.encode(request_data.prompt_text)

```

```

57     tokenized_data = TokenizedData(request_id=request_data.request_id, tokens=
        tokenized_prompt)
58
59     await self.pending_queue.put(tokenized_data)
60     return request_data.request_id

```

engine 侧拿到的对象从头到尾只有两个属性。它拿不到 `user_id`，也拿不到 `timestamp` ——不是因为有人拦着，是因为那个对象里根本没有。

为什么进热路径的那个故意不用 **pydantic**。pydantic 买到的是构造时的类型校验，加上 `model_dump_json()` / `model_validate_json()` 这一对（`inferenceLM/data/request.py:33--38`）；ADR-000 选它的理由就写了四个字，“自带 type checking”。问题是这两样好处对 `TokenizedData` 一个都不成立：它不过网络、不落盘、不序列化，生命周期是从 `submit_request` 里被造出来到 `LMEngine.inference()` 读完 `tokens` 为止，全程同一个进程、同一个 event loop。为它付一次 validation，买到的是零。而它是唯一穿过 queue 的对象，也就是单位时间被构造次数最多的类型——今天 QPS 是个位数所以看不出来，但把它做重没有理由。

这不是态度不一致，是分工：pydantic 出现在系统边界上，不出现在内部热路径上。判据不是“它是不是数据”，而是“它有没有一条通向外部的边”。

为什么重载 `__len__` 而不加一个 `length` 字段。ADR-000 明确拒掉了“加一个 `token length field`”，理由很短：“直接 overwritten `__len__` method 就可以了”。值得多看一眼的是这个决定的到达距离。`LMEngine.stopping_criteria()` 的第一个形参叫 `prompt_len`（`inferenceLM/engine/lm_engine.py:113`），而唯一的调用点（`inferenceLM/engine/lm_engine.py:105`）传的是 `len(tokenized_data)`——不是 `len(tokenized_data.tokens)`，是对象本身取长度。所以 `TokenizedData.__len__`（`inferenceLM/data/tokenized_data.py:19`）不是方便的小糖，是跨层契约的一部分：data 层的一个 dunder，被 engine 层的停止条件直接依赖。删掉它，坏的不是 data 层的测试，是 decode loop 的第一次迭代。这条线在 §3 继续。

字段	所属类	写入点	生产代码里的读取点
<code>user_id</code>	<code>RequestData</code>	receiver:45	无（只有 <code>__str__</code> 和测试）
<code>request_id</code>	<code>RequestData</code>	receiver:46	无（engine 读的是副本）
<code>timestamp</code>	<code>RequestData</code>	receiver:47	无（只有 <code>__str__</code> 和测试）
<code>status</code>	<code>RequestData</code>	receiver:48; engine:48,54,56	无（只有测试断言）
<code>prompt_text</code>	<code>RequestData</code>	receiver:44	receiver:56（喂给 tokenizer）
<code>max_token_length</code>	<code>RequestData</code>	receiver:49	engine:52
<code>do_sample</code>	<code>RequestData</code>	receiver:50	无人读取
<code>generated_tokens</code>	<code>RequestData</code>	engine:50（整体赋值）	无（只有测试断言）
<code>request_id</code>	<code>TokenizedData</code>	receiver:57	engine:48,50,52,54,56,57
<code>tokens</code>	<code>TokenizedData</code>	receiver:57	lm_engine:95
<code>__len__</code>	<code>TokenizedData</code>	——	lm_engine:105

Table 2: 两个数据类的字段与它们的真实读写点，行号逐个 grep 核对。receiver 指 `request_receiver.py`，engine 指 `inference_engine.py`。`timestamp` 和 `user_id` 没有读者不一定是错，它们留给 metrics 和多租户；`do_sample` 是另一回事，见下面的撤回框。

撤回 (retracted): Week 2 没有暴露任何缺少实现支撑的 interface

week2-retro §1.4 把“不提前暴露 `do_sample`”列为本周守住的一条 design discipline，理由写得很好：public interface 是 promise 不是 wishlist，暴露未实现的开关只会得到 silent contract violation 或 delayed failure。

代码里的事实是反过来的。`do_sample` 出现在 `RequestData`（`inferenceLM/data/request.py:27`）、`submit_request()` 签名（`inferenceLM/request\_receiver/request\_receiver.`

py:29) 和 LMEngine 的三个方法签名上。更糟的是它的下场: InferenceEngine 调 `lm_engine.inference()` 时只传了 `tokenized_data` 和 `max_length` (`inferenceLM/engine/inference_engine.py:50--53`), 没有传 `do_sample`。用户提交 `do_sample=True` 时, 字段被接收、被存、被序列化进 JSON, 然后在进入 engine 的那一刻静默丢弃, 拿到 greedy 结果——正好是 retro 自己点名的第一种失败模式。那两处 `NotImplementedError` (`lm_engine.py:45` 和 `70`) 在这条路径上永远不会触发。撤回的是“这周守住了”, 不是那条 discipline 本身。

Checkpoint 2.1 (two_data_classes): 把两个类合并回一个, 第一个坏掉的是什么

不看代码回答:

- (a) 把 `tokens` 直接挂到 `RequestData` 上、删掉 `TokenizedData`、`pending_queue` 里改放 `RequestData`。pipeline 还能跑通吗? 如果能, “坏掉”的是什么?
- (b) 合并之后 LMEngine 会新拿到哪些信息? 为什么这是个问题——它又不会去用。
- (c) §4 要做 continuous batching。合并之后, “batch 一个 request” 在内存里意味着什么?

Hint 先看 `pending_queue` 的类型标注写在哪一层。

参考答案: (a) 能跑通, 改动很小: `lm_engine.py:95` 一个字不用改, `len(tokenized_data)` 要改写。坏的不是运行, 是边界——从此 LMEngine 的入参里带着 `status`, “顺手更新一下 `status`” 就变成一行能写出来的事, 而 week2-retro §1.5 花了整节论证这件事不能做。

(b) `user_id`、`timestamp`、`status`、`prompt_text`、`do_sample`。“它不会去用”不是保护, 保护来自“它想用也拿不到”。前者靠纪律, 后者靠类型; 这个项目很多地方靠纪律 (`request_store` 就是), 这一处选了类型。

(c) 每个 batch slot 都拖着一个 pydantic 实例和原始 prompt 字符串, 而真正要拼进 tensor 的只有 `tokens`。不是不能做, 是每次组 batch 都要先把 `tokens` 摘出来——摘的动作本身就是在事后重建今天已经有的这个边界。

代码位置: `inferenceLM/data/tokenized_data.py:12`、`inferenceLM/engine/inference_engine.py:22`、`inferenceLM/engine/lm_engine.py:95`

Checkpoint 2.2 (pydantic_or_not): 一个用 pydantic 一个不用, 给两条不同的理由

不看代码回答: 说明这不是随手写成的不一致, 而是分工。

- (a) 第一条理由从“这个对象有没有通向进程外的边”出发。
- (b) 第二条理由从“这个对象在单位时间里被构造多少次”出发。
- (c) 如果要给 `TokenizedData` 加校验 (`token id` 必须非负), 应该加在哪里?

Hint 看 `RequestData` 有哪两个方法是 `TokenizedData` 完全没有的。

参考答案: (a) `RequestData` 有 `serialization()` / `deserialization()` (`inferenceLM/data/request.py:33--38`), 要被 dump 成 JSON 给外面看, 反序列化时数据来源不可信。`TokenizedData` 没有任何序列化方法, 所有实例都由同一段代码在同一进程里造出来。

(b) 每条 request 只造一个 `RequestData`, 但 `TokenizedData` 是唯一穿过 `pending_queue` 的类型, 数量随吞吐量放大。边界上的对象数量由用户决定, 热路径上的由吞吐量决定。

(c) 加在构造它的地方 (`Tokenizer.encode()` 的返回处或 `inferenceLM/request\_receiver/request\_receiver.py:57`), 而不是给 `TokenizedData` 套 `pydantic`。这个校验只在“文本变成 token”这一次转换处有意义, 之后每次传递重验一遍是纯浪费。校验属于转换点, 不属于类型。

代码位置: `inferenceLM/data/request.py:33--38`、`docs/decisions/adr000-request.md:52`

2.2 状态枚举与它的两个死值

`RequestStatus` 只有八行:

```
3 class RequestStatus(Enum):
4     PENDING = 0
5     PREFILLING = 1
6     DECODING = 2
7     DONE = 3
8     FAILED = 4
9
10    PROCESSING = 5 # week2 阶段不分 PREFILLING 和 DECODING
```

把 `PREFILLING` 和 `DECODING` 在仓库的 `.py` 与 `.md` 里 `grep` 一遍 (讲义自己的 `.tex` 不算), 命中九行: 枚举定义两行、`PROCESSING` 那行的注释一行、`ADR-001` 四行、`week2-retro` 一行、`CLAUDE.md` 一行。赋值点零处。实际跑起来的状态机是 `PENDING` → `PROCESSING` → `DONE` / `FAILED`, 转移点全在同一个函数里。而 `ADR-000` 的 `Consequences` 写的是“`pending` → `prefilling` → `decoding` → `completed`”: 中间两态不存在, 终态叫 `DONE` 不叫 `completed`, 实际存在的 `PROCESSING` 在 `ADR` 里根本没出现。

枚举值	数值	赋值点	说明
<code>PENDING</code>	0	<code>request.py:24</code> (默认值); <code>receiver:48</code>	唯一由 <code>receiver</code> 侧写入的状态
<code>PREFILLING</code>	1	从未	只在枚举定义里存在
<code>DECODING</code>	2	从未	只在枚举定义里存在
<code>DONE</code>	3	<code>engine:54</code>	<code>inference()</code> 正常返回后
<code>FAILED</code>	4	<code>engine:56</code>	<code>except Exception</code> 分支
<code>PROCESSING</code>	5	<code>engine:48</code>	后加的, 所以数值排在 <code>FAILED</code> 之后

Table 3: `RequestStatus` 六个成员的实际赋值点。成员数是六个不是五个——`PROCESSING` 是 Week 2 补进来替代中间两态的, 注释就写在 `inferenceLM/data/request\_status.py:10`。

教学点不是“枚举该怎么写”, 是文档和代码不一致会在类型系统里留下什么形状的疤。这里有三道。

第一道在穷尽性上。任何一处想覆盖全部 `status` 的分支 (今天还没有, 写 `metrics` 导出或 §5 讲的差分测试时一定会有) 都被迫处理两个不可达分支: 写了是死代码, 不写则类型检查器会报 `non-exhaustive`。这个成本每新增一个 `status` 消费者就重付一次。

第二道在线上格式里。`pydantic` 序列化 `Enum` 用值不用名字, 实测 `RequestData.serialization()` 输出的是 `"status":3` (`tests/request_test.py:71` 断言的是 `"status":0`)。于是对外暴露的状态码是 `{0,3,4,5}`: 中间空着 1 和 2, 而 5 排在 4 后面。下游必须拿着枚举表才能解释, 且会合理地怀疑自己漏了两种状态。`PROCESSING = 5` 排在最后, 把“它是补上去的”这段历史直接写进了 `wire format`。

第三道在文档的约束力上, 作者自己诊断过。`week2-retro` 第 3 节把这条单列为“问题 3: `ADR` 和代码不一致”, 后果是“文档失去约束力, 最后变成历史遗迹”和“自己对 `system` 的 `mental model` 被分裂成两套”, 举的 `concrete example` 正是这个枚举 (`docs/retro/week2-retro.md:682`):

“如果 `ADR` 写 `PREFILLING` → `DECODING` → `DONE`, 但 Week 2 实际实现只是 `PROCESSING` → `DONE`, 那就要明确写出来: `Week 2 collapses prefill/decode into PROCESSING; fine-grained states deferred to Week 3 because no scheduler exists yet.`”

诊断对，处方也对。但这句话写在 retro 里，没有写回 ADR——今天去读 `adr000-request.md`，Consequences 仍是那条四态链，Status 仍是 Accepted。这类修复的真实成本就在这里：诊断几乎免费，同步不是。

Checkpoint 2.3 (dead_enum_values): 两个从未被赋值的枚举成员，是 bug 还是设计

不看代码回答：

- (a) 按 week2-retro §1.4 自己定的规则（“if abstraction solves no current problem: remove it”），PREFILLING 和 DECODING 该不该删？
- (b) 如果不删，最小的补救动作是什么？落在代码里还是文档里？
- (c) §4 实现 continuous batching 之后，这两个值会自动变得有意义吗？

Hint 先分清“这个值今天有没有 execution path”和“有没有 consumer”——不是同一个问题。

参考答案：(a) 按字面该删，但要注意 §1.4 那条规则针对的是 public interface（do_sample 是暴露给用户的开关，暴露即承诺），而这两个是内部状态标签，没有用户会去设置，撒不了谎。它更接近死代码，代价是认知负担和穷尽性检查，不是 contract violation。
 (b) 落在文档：把 ADR-000 的 Consequences 改成实际的四态链，加一句“PREFILLING/DECODING reserved for Week 3, no execution path today”。这两个值不影响运行时行为，问题在于读代码的人会以为它们在用。inferenceLM/data/request_status.py:10 的注释已经做了一半——解释了 PROCESSING 为什么存在，没解释另外两个为什么留着。
 (c) 不会自动。需要先有一个读者：scheduler 要按 stage 分组，“这条 request 现在在哪一阶段”才第一次成为要被查询的问题。今天 LMEngine.inference() 内部其实已经分了 prefill 和 decode 两段，但没人需要从外面知道它在哪段。

代码位置：inferenceLM/data/request_status.py:5--6、docs/decisions/adr000-request.md:65、docs/retro/week2-retro.md:682

2.3 一个类变量当全局计数器

request_id 是这么来的：index = 0 写在 `class RequestReceiver` 的类体里（inferenceLM/request_receiver/request_receiver.py:22），是类变量不是实例属性；读在 inferenceLM/request_receiver/request_receiver.py:46，加在 inferenceLM/request_receiver/request_receiver.py:53。两处都显式写 `RequestReceiver.index` 而不是 `self.index`——后者会在第一次赋值时悄悄创建实例属性，把类变量遮住，计数器就变成 per-instance 的了。这里写对了。

买到的是进程内单调递增、无重复的 id，代价是零同步。这不是运气，是单线程 event loop 给的：submit_request 虽然是 async def，但从第 46 行读 index 到第 53 行 += 1 之间没有任何 await，这段 read-modify-write 相对于 event loop 是原子的。换成线程池，+= 不是原子操作，这里就得上锁。

这个不变量是隐式的——源码里没有一句注释说“这几行之间不能加 await”。第 56 行的 tokenize 排在 index += 1 之后读起来像随手的顺序，实际上被这个不变量约束着：哪天 Tokenizer.encode() 变成 async（换成远程 tokenizer service），只要那个 await 挪到 46–53 之间，十个并发 submit_request 就会拿到重复 id，而且是概率性的、测试里不一定复现。

代价是跨实例共享：类变量属于类，同一进程里两个 RequestReceiver 共用一个计数器。这个代价在测试里立刻显形（见下面的 checkpoint）。生产上的代价还没到，但方向清楚：多进程部署时 id 会撞，进程重启后从 0 重新发号。今天不持久化、单进程，所以没爆。

Checkpoint 2.4 (class_var_index): 类变量计数器如何逼着测试改写法

不看代码回答：

- (a) tests/request_receiver_test.py 里三个测试各新建一个 RequestReceiver 并并发提交 10 条。第二个测试拿到的 request_id 是 "0"-"9" 吗？
- (b) 测试因此不能硬编码期望 id。指出具体是哪一行承担了这个补偿，以及为什么它必须在 submit_request 之前执行。
- (c) 把 index 改成实例属性，这三个测试会变绿还是变红？算修好了吗？

Hint 三个测试共用 module 级的 queue 和 store（该文件第 5-6 行），但那不是关键——第三个共享物根本不在那两行里。

参考答案: (a) 不是。第一个测试已经把计数器推到 10，第二个拿到 "10"-"19"，第三个是 "20"-"29"；实际值还取决于 pytest 的收集顺序和同 session 里别的文件有没有先构造过 RequestReceiver，这让它更脆。

(b) 是 tests/request_receiver_test.py:16 这一行先把计数器读出来当基准：

```
16 request_idx = RequestReceiver.index
17 request_ids = [f"{request_idx + i}" for i in range(10)]
```

同样的写法在 tests/request_receiver_test.py:54 再出现一次。必须在 submit_request 之前执行，因为它会把计数器推走，读晚了基准就偏 10。tests/data_flow_test.py:82 有一句注释记了同一个坑（那句里“static method”的说法不准确，它是类变量）。

(c) 会变绿，还能改回硬编码。但没修好：实例属性意味着两个 receiver 发出重复 id，而 request_store 是共享的，后写的会直接覆盖前一个（inferenceLM/request_receiver/request_receiver.py:52 是无条件赋值，没有查重）。测试更绿、系统更坏。真正的修法是 UUID 或 (worker_id, counter)。

代码位置： inferenceLM/request_receiver/request_receiver.py:22、tests/request_receiver_test.py:16、tests/request_receiver_test.py:54

2.4 整个系统只靠两个共享可变对象耦合

Week 2 的系统有两个模块，request_receiver 和 engine。它们之间传数据、传状态、传完成信号，而两个源文件互不 import——这一点值得亲手核实，它是整个分层的支点。

```
# request_receiver.py:1-7  asyncio / time / TokenizedData / RequestData
#                          / Tokenizer / RequestStatus / typing.Dict
# inference_engine.py:1-9  asyncio / RequestStatus / TokenizedData
#                          / typing.Dict / RequestData / transformers / LMEngine
```

两份清单的交集是 inferenceLM/data/ 下的三个类，没有一边 import 另一边；四个 __init__.py 全是空文件，所以也没有包级别的偷偷连线。（ADR-000 的 NOTE 说“Request 层和 Engine 层都需要 import Request Class 和 TokenizedData Class”，这句是对的，但常被读成“两层互相依赖”——实际是两层共同依赖 data 层。）

连起来靠两个对象，构造时从外面注入两端：

```
pending_queue = asyncio.Queue()
request_store = {}
receiver = RequestReceiver("gpt2", pending_queue, request_store)
inference_engine = InferenceEngine(model, pending_queue, request_store)
```

这四行是 `tests/data/_flow/_test.py:20--23` 的原样。今天没有 `main()` 把它们串起来——根目录的 `main.py` 只打印一句 `hello`，所以测试就是唯一的 `composition root`。

`pending_queue` 兼三个职责：数据传递、背压（`Queue()` 无参数所以今天无界，背压其实没生效）、以及完成信号（`join()` 配 `task_done()`，§3 会拆）。`request_store` 是普通 `dict`：`receiver` 写入新条目（`inferenceLM/request/_receiver/request/_receiver.py:52`），`engine` 就地改 `status` 和 `generated_tokens`（`inference_engine.py:48, 50, 54, 56`）。两个 `writer`、一个 `dict`、没有锁，理由和 `index` 那节一样。

代价：耦合没有消失，只是换了地方。把依赖从 `import graph` 挪到 `runtime object graph`，好处是 `import` 期看不到彼此，坏处是类型检查器也看不到。今天没有任何代码或类型标注能保证两边拿到的是同一个 `queue`；传错了不会报错，只表现为“提交了但永远不被处理”，而且因为 `join()` 一直挂着，症状是测试超时而不是断言失败。两边对同一个对象的标注还不一致：`engine` 侧是 `asyncio.Queue[TokenizedData]`（`inferenceLM/engine/inference_engine.py:22`），`receiver` 侧是光秃秃的 `asyncio.Queue`（`inferenceLM/request/_receiver/request/_receiver.py:19`）。

还有一个没人管的问题：`request_store` 只增不删，请求做完后条目连同 `prompt` 文本一直留到进程结束。`BACKLOG.md` 的 P0 有一条讲 `engine lifecycle` 和潜在 `memory leak`，但说的是 `dangling task` 和 `queue drain`，没有一条记着 `store` 的清理。这是“共享可变状态没有明确 `owner`”的典型后果：两个模块都写，所以谁都不觉得该负责删。

Example 2.1 (request_lifecycle_trace): 一条 request 的四个时刻

下面是一条真实 `request` 的 `RequestData` 快照，用 `gpt2` 实跑得到（挂一个 `watcher coroutine` 在每次 `status` 变化时打印）。`prompt` 是 “Yifan Yu is ”，`max_length=20`。

`RequestReceiver.index` before submit = 0 / after submit = 1

```
[T1 submit_request 返回] status=PENDING      generated_tokens=[]  qsize=1
[T2 engine 取走]        status=PROCESSING   generated_tokens=[]  qsize=0
[T3 inference() 返回]    status=DONE         generated_tokens=[1849, 64,
                    1849, 35465, 1849, 805, 508, 468, 587, 2877, 287, 262, 1748, 329]
[T4 join() 返回]        与 T3 完全相同
```

```
timestamp=1785311490.085415 max_token_length=20 do_sample=False user_id='user_0'
len(prompt tokens)=6         len(generated_tokens)=14
```

T1 到 T2 之间 `RequestData` 一个字节都没变，但 `qsize` 从 1 变成 0。`request` 的“位置”根本不记录在 `RequestData` 里——它在不在 `queue` 里是 `queue` 的事。这是两个类拆开的直接结果。

T2 和 T3 之间没有中间态。`generated_tokens` 在整个 `PROCESSING` 期间一直是 `[]`，然后一次性变成 14 个。原因在 `inferenceLM/engine/inference_engine.py:50`：整个 `inference()` 的返回值整体赋值给字段，不是逐 `token` 追加。所以尽管 `inferenceLM/data/request.py:28` 的注释写着“提前准备 `buffer`”，它今天不是流式 `buffer`，只是终值槽——今天做不了 `streaming` 输出，不是因为缺一个 `endpoint`，是因为中间结果压根没有出口。

$6 + 14 = 20$ 。`max_token_length` 管的是 `prompt` 加生成的总长，语义写在 `inferenceLM/data/request.py:26` 的注释里，由 `stopping_criteria` 强制（`inferenceLM/engine/lm_engine.py:119`）。500 `token` 的 `prompt` 配 `max_length=20` 不会生成 20 个 `token`，会直接 `raise`。

序列化出来是（注意 `status` 是数字 3，不是 “DONE”）：

```
{"user_id": "user_0", "request_id": "0", "timestamp": 1785311490.085415, "status": 3,
 "prompt_text": "Yifan Yu is ", "max_token_length": 20, "do_sample": false,
 "generated_tokens": [1849, 64, ..., 1748, 329]}
```

Checkpoint 2.5 (shared_mutable_coupling): 两个共享可变对象是不是一种耦合

不看代码回答：

- (a) 两个模块互不 import，是否说明它们之间没有耦合？如果有，耦合具体写在哪里？
- (b) 把 request_store 换成 Redis 要改几行？换 pending_queue 呢？为什么成本差这么多？
- (c) 有没有任何机制保证两边拿到的是同一个 queue？传错了症状是什么？

Hint 数一数一共有几处代码会写 request_store，分别在哪个模块里。

参考答案：(a) 耦合有，只是不在 import graph 上，而写在两个对象的隐式契约里：queue 的元素必须是 TokenizedData；store 的 key 必须是同一个 request_id；store 里的条目必须先于 engine 取到它而存在。最后这条尤其脆——engine 在 inferenceLM/engine/inference_engine.py:48 直接下标取值，没有 .get() 兜底，缺了就是 KeyError，而这个 KeyError 会被 except Exception 吞掉变成一条 print。三条契约没有一条写在类型标注里。

(b) 换 store 要改 5 处 (receiver:52, engine:48/50/52/54/56)，而且要处理“值不再是同一个对象引用、改完要写回”——今天 engine 是就地修改 pydantic 对象。换 queue 表面只有 put 和 get 两处，但 join()/task_done() 这套完成信号语义大多数消息中间件不提供，所有测试的等待方式都要重写。差别在于一个是数据结构，一个还兼着控制流。

(c) 没有。传什么完全由调用方决定 (tests/data_flow_test.py:20 是唯一的 composition root)。传错了不报错也不抛异常，症状是 await pending_queue.join() 永远不返回——测试挂死而不是断言失败，最难读的一类失败。

代码位置： inferenceLM/engine/inference_engine.py:26--35、inferenceLM/request_receiver/request_receiver.py:24--27、tests/data_flow_test.py:20

Downscaling Tip: 用 dummy token 绕开 tokenizer

tests/inference_engine_test.py 里跑 10 条 request 的那个测试压根没有构造 RequestReceiver，直接拿整数当 token：

```
32     for i in range(10):
33         user_input = f"Prompt {i}"
34         input_ids = [i] # dummy tokenized data, just use the index as the token...
```

TokenizedData 对 tokens 没有任何校验（它是裸 class，见 §2.1），所以 [3] 是一个完全合法的一 token prompt。说清楚它省掉的是什么：省掉的是 tokenizer，不是模型——同文件的 module 级 fixture 仍然 GPT2LMHeadModel.from_pretrained("gpt2") (tests/inference_engine_test.py:14--16)，权重该下还是要下。真正的收益是测试的语义变干净：它验的是 engine 的取件-处理-回写循环，不会因为 tokenizer 版本变化而失败。

Resource requirements: CPU 即可，不需要 GPU/MPS。实测 pytest tests/request_receiver_test.py tests/inference_engine_test.py -q 在本机 (Apple Silicon, CPU 路径，权重已缓存) 是 7 passed in 25.79s。

这一节定下来的东西——两个数据类、一个进程内计数器、两个共享可变对象——在 Week 2 的规模下几乎不产生可观测的差别。价值全在下游：§3 会看到 LMEEngine 之所以能被写成 pure compute unit，正是因为它的入参里没有 status 可改。边界是先划出来的，不是后来重构出来的。

3 手写 inference loop: prefill / decode 与 Weak Async

三行就能生成文本：

```
inputs = tokenizer("Yifan Yu is ", return_tensors="pt")
outputs = model.generate(**inputs, max_length=20)
print(tokenizer.decode(outputs[0]))
```

这三行是对的、快的、边界情况处理得比我们好的。那为什么这个项目里有一个手写 prefill、手写 decode 循环、手写停止条件的 LMEngine? week2-retro §1.1 把理由压成一句话 (docs/retro/week2-retro.md:48):

”High-level API 的代价，不是‘少一点灵活性’，而是你会失去整个 backend architecture 的 control point。”

重点在最后三个字。不是”我调不了 temperature”——那种参数 generate() 都有。是在两个 step 之间插入代码的能力：decode 走完第 5 步、第 6 步之前，我想看一眼 queue 里有没有新 request，想把这一步的 KV cache 挪个地方，想决定这条 request 要不要被抢占。generate() 把整个 prefill → decode 循环封在里面，只在结束时交出 output ids。循环体不归你，插入点就不存在。

3.1 为什么不用 generate()

ADR-001 的 rejection reason 写在 docs/decisions/adr001-inference.md:27: ”它不暴露中间状态 (KV cache、per-step logits)，导致你无法在 Week 3/4 实现 continuous batching 和 paged KV cache。” week2-retro §1.1 把”中间状态”展开成五项：per-step logits、updated past_key_values、prefill / decode 的 boundary、step-level scheduling point、以及未来做 speculative decoding 或 paged KV cache 时对中间状态的 access (docs/retro/week2-retro.md:37--42)。

前两项是数据，后三项是控制。数据那两项 generate() 其实给得出一部分 (output_scores=True)，但那是事后的一份记录，不是在循环里当场据此做决定。控制那三项才真拿不到：LogitsProcessor 和 StoppingCriteria 能改这一步的 logits、能让这条序列停，但改不了谁在这个 batch 里、下一步跑哪些序列——而 continuous batching 要改的正是后者。

顺带纠一个方向：手写不是性能优化。本仓库把两条路径放在一起跑的地方有两处 (tests/lm_engine_test.py 和 tests/transformer_lm_adapter_test.py)，比的都是输出是否逐 token 相同，没有一处比过时间。真要比，手写版每步还多一次 .item() (inferenceLM/engine/lm_engine.py:102 和 :108) 也就是每步一次同步，而 generate() 把整条序列留在 device 上最后一次性搬回。单条 request 下手写版更可能更慢。它换来的是控制权，是一笔当下亏损、指望在 batching 之后回本的交易。

代价：那些 edge case 现在归你了。ADR-001 列 generate() 的”好处”时自己写了：代码量少、不需要手动管理 KV cache、不需要自己写 stopping logic、HuggingFace 已经帮你处理了 edge case (docs/decisions/adr001-inference.md:28)。最后这条是手写方案的债务，余额可以精确读出来——stopping_criteria() 一共七行，判据两条：

```
113 def stopping_criteria(self, prompt_len: int, generated_tokens: List[int], max_length:
114     int) -> bool:
115     """
116     Return True (means stop) if:
117         1. generated_tokens reaches max length (e.g. 20)
118         2. the last generated token is the EOS token.
119     """
120     return len(generated_tokens) + prompt_len >= max_length or generated_tokens[-1]
121     == self.model.config.eos_token_id
```

达到长度上限，或最后一个生成的 token 是 EOS。就这两条。generate() 在同一位置还支持 min_new_tokens、任意 stop string、max_time、自定义 StoppingCriteriaList；再往外还有 repetition_penalty、no_repeat_ngram_size、bad_words_ids，以及 batch 内不等长序列的 attention mask 和 padding。今天一条都没有。

还有三处细节，都是“自己写”必然撞上的那类问题。generated_tokens[-1] 在列表为空时会 IndexError，今天不会发生只是因为 prefill 一定先 append 一个 token (inferenceLM/engine/lm_engine.py:102) ——安全性来自调用顺序而非函数自身。EOS 被 append 之后才停，所以返回值包含 EOS；和 generate() 一致，但它是巧合还是有意对齐，代码里没写。还有一处 guard 重复：inference() 检查等于 max_length (inferenceLM/engine/lm_engine.py:98)，prefill() 又检查大于等于 (:32)，判据不等价、文案不同；名字里带 prefill_should_reject 的那个测试 (tests/lm_engine_test.py:55) 断言的其实是 inference() 那条的文案，所以 prefill() 自己那条 guard 今天没有测试覆盖。

Checkpoint 3.1 (why_not_generate): 手写 loop 换到了什么，又欠下了什么

不看代码回答：

- (a) generate() 提供 StoppingCriteria 和 LogitsProcessor 两种回调。有了它们，为什么还是说“拿不到 step-level scheduling point”？
- (b) 再说出三条 generate() 支持而这里没有的停止/约束能力。
- (c) 只做单条 request 的文本生成，手写版更快还是更慢？为什么？

Hint 区分“能不能改这一步的输出”和“能不能改下一步跑哪些序列”。

参考答案：(a) 因为回调的输入是当前这一步的 logits 和已生成序列，输出是修改后的 logits 或一个 bool。它们改不了 batch 的成员：没法在回调里把新 request 塞进当前 batch、把低优先级的踢出去、或决定下一次 forward 的 input_ids 是什么形状。scheduling point 需要的是循环体的所有权，回调只是循环体里的一个洞。

(b) 任选三条：min_new_tokens、stop string、max_time、repetition_penalty 或 no_repeat_ngram_size、bad_words_ids、batch 内不等长序列的 mask 与 padding。

(c) 大概率更慢：手写版每 token 做一次 .item() (lm_engine.py:102 和:108)，那是一个 device→host 同步点；generate() 结束后才一次性取回。换来的是控制权，这笔交易在 batch size 变大之前是净亏的。

代码位置：docs/decisions/adr001-inference.md:26--28、docs/retro/week2-retro.md:37--48、inferenceLM/engine/lm_engine.py:113--119

3.2 KV cache 为什么合法

先划边界：这一章不实现 KV cache。这份代码里没有一行分配、拷贝、分页或驱逐缓存，它只做一件事——把上一次 forward 吐出来的 outputs.past_key_values 原样传回给下一次 (inferenceLM/engine/lm_engine.py:40、:60、:64、:106)。缓存由 HuggingFace 管。缓存的数据结构长什么样、怎么分块、怎么跨 request 复用和回收，是姊妹篇的内容。这里只讲凭什么可以缓存。

常见误解：KV cache 之所以成立，是因为过去的 token 不会变

已经生成的 token 是不变的，所以它们对应的 K 和 V 也不变，存下来复用就行了。

这句不能说错，但它不构成解释——是把结论换了个说法。week2-retro §2.1 自己点了这条，并追问了真正的问题：为什么过去 token 的 K/V 不需要重算？答案是 causal mask。在 decoder-only

transformer 里位置 i 只能看到 $0 \dots i$, 写成 dependency chain:

```
h[l][i] = f( h[l-1][0..i] )      # 第  $\ell$  层位置  $i$ , 只依赖下层左边
K[l][i] = W_k @ h[l-1][i] ; V[l][i] = W_v @ h[l-1][i]
```

```
# 追加新 token  $n+1$  时: 对任意  $i \leq n$ ,  $h[l][i]$  的输入集合没有变化
#  $\Rightarrow h[l][i]$  不变  $\Rightarrow K[l][i], V[l][i]$  不变
```

关键在中间那步, 而且它必须逐层成立——这正是 retro §2.1 特意提醒的地方 (“注意 KV Cache 有很多层”)。只在最后一层论证会漏掉一个可能性: 新 token 会不会通过某个下层绕上来影响旧位置? 答案是不会, 因为 causal mask 在每一层都成立, 性质沿层向上传递。把它讲成归纳, 比讲成 “过去的不变” 扎实得多。

层数不是抽象的。GPT-2 small 有 12 层, 实测:

```
>>> out = model(input_ids=torch.tensor([[15496, 11, 616]]), past_key_values=None)
>>> type(out.past_key_values), len(out.past_key_values)
(<class 'transformers.cache_utils.DynamicCache'>, 12)
>>> out.past_key_values[0]
TypeError: 'DynamicCache' object is not subscriptable
```

最后一行值得留意。lm_engine.py 给 KV 写的类型标注是 List[Tuple[torch.Tensor, torch.Tensor]] (inferenceLM/engine/lm_engine.py:39、:53、:63), 那是 HuggingFace 的旧缓存格式; 本仓库锁的 transformers>=5.5.3 上流过来的实际是 DynamicCache, 能 len() 但不能下标。运行时不受影响 (标注不强制, 而这份代码对它只做 “接住、原样传回”), 但这个不一致本身说明: 我们对这个对象的了解只到 “能原样传回去” 为止。要往里看、按 block 切、跨 request 复用, 就必须先撕开这层抽象——那是姊妹篇在做的事。

Checkpoint 3.2 (kv_cache_legality): 把 “KV cache 为什么成立” 讲成 dependency chain

不看代码回答:

- (a) 用一句话解释合法性, 句子里必须出现 causal mask, 不能出现 “过去的 token 不会变”。
- (b) 为什么 “只在最后一层论证” 不够?
- (c) encoder-only 模型 (BERT) 能不能用同样的 KV cache? 为什么?

Hint 先问 “位置 i 的第 ℓ 层输入来自哪些位置”, 再问这个答案会不会因为序列变长而改变。

参考答案: (a) 参考说法: causal masking 保证任何位置只 attend 到自己和左侧, 追加新 token 不会进入既有位置的可见集合, 所以先前算出的 K/V 仍然有效。retro §2.1 的英文版是 “Because causal masking prevents earlier tokens from depending on future tokens, previously computed K/V remain reusable when new tokens are appended.”

(b) 因为 K/V 每层各一份 (GPT-2 small 12 份)。要断言第 12 层旧位置的 K/V 不变, 得先断言第 11 层旧位置的输出不变, 一路递归到 embedding。没有这一步, “新 token 从下层绕上来影响旧位置” 在逻辑上没有被排除。

(c) 不能。BERT 是双向注意力, 往后加一个 token, 前面所有位置的上下文都变了, K/V 全部失效。这正说明合法性来自 mask 的形状, 而不是 “token 值不变”——BERT 的旧 token 值同样没变。

代码位置: docs/retro/week2-retro.md:331--379、inferenceLM/engine/lm_engine.py:60

3.3 shape 是一个 systems concern

prefill 和 decode 最省事的区分是“一个读 prompt，一个生成 token”。这话对，但推不出系统结论。retro §2.2 换成 input shape，结论就自己掉出来：prefill 是 [batch, seq_len]，decode 是 [batch, 1]。

在这份代码里 batch 恒等于 1，两个 shape 分别造于 inferenceLM/engine/lm_engine.py:95 (unsqueeze(0) 把 token 列表变成 (1, seq_len)) 和 :43/:67 (argmax(...).unsqueeze(1) 产出 (1, 1))。注意后者：decode 下一步要吃的 [B,1] 就是上一步 argmax 的输出直接 unsqueeze 出来的，中间不经过任何拼接——“decode 每次只喂一个 token”不是一条策略，是 shape 自然导出的。

输出侧更能说明问题。实测 prefill (3 个 prompt token) 的 logits.shape 是 [1, 3, 50257]，decode 是 [1, 1, 50257]。prefill 那次算出了全部三个位置的 logits，而代码只取最后一个 (inferenceLM/engine/lm_engine.py:38)。前两个被直接丢掉——它们不是没用（训练时 teacher forcing 要的正这些），但在推理路径上是纯浪费，prompt 越长浪费越多。

由此掉出三条系统结论，都会在 §4 兑现：decode step 之间天然好 batch (n 条的输入都是 [1,1]，堆起来就是 [n,1])；prefill 和 decode 混在一个 forward 里不自然（一条要 [1,S] 一条要 [1,1]， S 还各不相同，要塞进同一个 forward 就得 padding 或 ragged attention，而 docs/progress/week3-discussion.md 记的正是 HF 的 model() 不支持 packed/ragged，所以 batching 必须 stage-homogeneous）；chunked prefill 是真实优化点，因为它把那个大而不断的 S 切成 scheduler 能管理的固定单位。

还有一条常被一起讲、但不是本仓库实测的结论，这里标明来源：prefill 算术强度高、通常 compute-bound，decode 一次只算一个位置却要读全部权重、通常 memory-bandwidth-bound。这是社区的标准结论，本项目没有做过 roofline 或带宽测量来验证，讲义在这里只是引用。

Checkpoint 3.3 (prefill_decode_shape): shape 差异会传导到哪些设计决定

不看代码回答：

- (a) 写出两次 forward 的 input_ids 和 logits shape (batch=1, prompt 长度 S)。
- (b) prefill 算出来的 logits 有多少被用上？剩下的在别的场景下有用吗？
- (c) 为什么“把一条 prefill 和一条 decode 拼进同一个 forward”比“把两条 decode 拼在一起”难得多？

Hint 第三问先写出两种情况下拼出来的 tensor 形状，答案就在形状里。

参考答案：(a) prefill: [1, S] 和 [1, S , vocab] (实测 [1, 3, 50257])。decode: [1, 1] 和 [1, 1, vocab]。

(b) 只有最后一个 (lm_engine.py:38 的 [:, -1, :1])， S 个里丢掉 $S-1$ 个。它们在训练时每个都是一个监督信号；推理时除了最后一个之外都对应已知 token。speculative decoding 是例外——验证草稿 token 要的正这些中间位置的 logits。

(c) 两条 decode 拼起来是 [2, 1]，形状齐整。prefill 加 decode 需要 [2, max(S , 1)]，短的要 padding，而 padding 位置的 K/V 会污染 cache，还得配 mask；两条 prefill 的 S 不同也一样。真正的解法是 ragged/packed attention，HF 的 model() 不提供，所以本项目在 §4 采用 stage-homogeneous batching。

代码位置：inferenceLM/engine/lm_engine.py:38、:43、:95、docs/retro/week2-retro.md:383--432

3.4 四个方法逐行读

prefill() 的执行体做三件事：守卫、一次 forward、一次 argmax。

32 | `if input_ids.shape[1] >= max_length:`

```

33         raise RuntimeError(f"Input token sequence length {input_ids.shape[1]} exceeds
max_length {max_length}.")
34
35     with torch.no_grad():
36         outputs = self.model(input_ids=input_ids, past_key_values=None)
37         new_logits: Float[tuple(torch.Tensor, "1, Vocab")]
38         new_logits = outputs.logits[:, -1, :]
39         new_kv: List[tuple(Float[tuple(torch.Tensor, "...")], Float[tuple(torch.Tensor, "...")])]
40         new_kv = outputs.past_key_values
41
42     if not do_sample:
43         new_token = torch.argmax(new_logits, dim=-1).unsqueeze(1) # shape (1, 1)
44     else:
45         raise NotImplementedError(f"Sampling strategy {do_sample} not implemented.")
46
47     return new_token, new_kv

```

`torch.no_grad()` 包住 `forward` (第 35 行) 是必须的, 不然每步都建计算图、显存单调上涨。第 43 行产出 (1, 1), 正好是 `decode` 要吃的形状。`decode()` 几乎是同一段代码:

```

59     with torch.no_grad():
60         outputs = self.model(input_ids=latest_token, past_key_values=latest_kv,
use_cache=True)
61         new_logits: Float[tuple(torch.Tensor, "1, Vocab")]
62         new_logits = outputs.logits[:, -1, :]
63         new_kv: List[tuple(Float[tuple(torch.Tensor, "...")], Float[tuple(torch.Tensor, "...")])]
64         new_kv = outputs.past_key_values
65
66     if not do_sample:
67         new_token = torch.argmax(new_logits, dim=-1).unsqueeze(1) # shape (1, 1)
68         return new_token, new_kv
69     else:
70         raise NotImplementedError(f"Sampling strategy {do_sample} not implemented.")

```

唯一的实质差别在第 60 行: `past_key_values=latest_kv`, 而且显式传了 `use_cache=True`。回头看 `prefill` 第 36 行——`past_key_values=None`, 没有传 `use_cache`。这是一处真实的不对称, 留到 `checkpoint` 里讲。串起两者的是 `inference()`:

```

88     # Check if the max_len is gerater than model's official max_len
89     model_capacity = self.model.config.n_positions
90     if max_length > model_capacity:
91         raise RuntimeError(f"User defined max_length {max_length} exceeds the model's
official maximum context length {model_capacity}.")
92
93     # Generated Tokens' buffer
94     generated_tokens = []
95     token_tensor = torch.tensor(tokenized_data.tokens).unsqueeze(0) # shape (1,
seq_len)
96
97     # Prefill step
98     if token_tensor.shape[1] == max_length:
99         raise RuntimeError(f"Input token sequence length {token_tensor.shape[1]}
equals max_length {max_length}, no space for generation.")
100     latest_token, latest_kv = self.prefill(token_tensor, do_sample=do_sample,
max_length=max_length)
101     await asyncio.sleep(0) # 先做 0 秒钟的别的事, 等一下 prefill
102     generated_tokens.append(latest_token.item())
103

```

```

104     # Decode step (用一个 cls.method() 来检查是不是 halt 生成)
105     while not self.stopping_criteria(len(tokenized_data), generated_tokens,
max_length):
106         latest_token, latest_kv = self.decode(latest_token, latest_kv, do_sample=
do_sample)
107         await asyncio.sleep(0) # 先做 0 秒钟的别的事, 等一下 decode
108         generated_tokens.append(latest_token.item())
109
110     return generated_tokens

```

按顺序看四处。第 89–91 行拿 `model.config.n_positions` 当上界，是唯一一处 LMEngine 读模型配置做校验、也是唯一能拦住“用户要 2048 但模型只有 1024”的地方。第 95 行没有指定 `device`：今天所有测试在 CPU 上跑所以没暴露，模型一旦 `.to("cuda")`，这行就会造出 CPU tensor 喂给 GPU 模型。第 102 行的 `.item()` 是每一步的同步点。第 105 行的 `stopping_criteria(len(tokenized_data), ...)` 就是 §2.1 埋的那条线——`__len__` 重载在这里兑现。

Checkpoint 3.4 (use_cache_asymmetry): prefill 没传 use_cache, decode 传了

不看代码回答：

- (a) 两个方法调 `self.model(...)` 时各传了哪些参数？差在哪一个？
- (b) 既然 prefill 没要求返回 cache，为什么下一行还能取到东西而不是 None？
- (c) 这是 bug 吗？如果不是，它错在哪一层？

Hint 去查 GPT2Config 里 `use_cache` 的默认值，再想“默认值兜底”和“显式声明意图”是不是同一件事。

参考答案：(a) prefill: `input_ids=input_ids, past_key_values=None`，没有 `use_cache`。decode: `input_ids=latest_token, past_key_values=latest_kv, use_cache=True`。
(b) HuggingFace 的 config 默认就是开的：实测 `GPT2LMHeadModel.from_pretrained("gpt2")` 之后 `model.config.use_cache` 是 `True`，不传等价于传 `True`，`outputs.past_key_values` 照样是一个 `DynamicCache`（实测 `len` 为 12）。这段代码今天正确，靠的是默认值兜底。
(c) 不是 bug——行为上没问题，`tests/lm_engine_test.py` 三个 case 逐 token 对上 `generate()` 就是证据。它错在意图表达：同一个类的两个姊妹方法对同一个参数一个说一个不说，读者只能推断“大概 prefill 不需要 cache”，而事实恰恰相反——prefill 产出的 cache 正是 decode 全部工作的前提。风险也真实：换一个 `use_cache` 默认为 `False` 的配置（从训练 checkpoint 加载时很常见），prefill 会安静地返回 `None`，然后在 decode 第一步炸掉，报错位置离病因隔了一个函数。加上 `use_cache=True` 是一个字符的修复。

代码位置：inferenceLM/engine/lm_engine.py:36（无）与 inferenceLM/engine/lm_engine.py:60（有）

Example 3.1 (greedy_matches_hf): 手写 loop 与 generate() 逐 token 相同

这是本仓库对“手写没写错”的核心证据。greedy 是确定性的，所以两条路径应当输出完全相同的 token 序列——不是“意思差不多”，是列表相等。ADR-001 把这一点写成了选 greedy 的理由之一。实跑（gpt2, CPU, prompt = "Yifan Yu is ", max_length=20）：

```

>>> input_ids[0].tolist()
[56, 361, 272, 10605, 318, 220] # 6 个 prompt token
>>> hf_out = model.generate(input_ids, do_sample=False, max_length=20,

```

```

...         pad_token_id=tok.eos_token_id, eos_token_id=tok.eos_token_id)
>>> hf_new = hf_out[0].tolist()[input_ids.shape[1]:] # 必须切掉 prompt
[1849, 64, 1849, 35465, 1849, 805, 508, 468, 587, 2877, 287, 262, 1748, 329]

>>> td = TokenizedData(request_id="r0", tokens=input_ids[0].tolist())
>>> eng_new = await LMEngine(model).inference(td, do_sample=False, max_length=20)
[1849, 64, 1849, 35465, 1849, 805, 508, 468, 587, 2877, 287, 262, 1748, 329]
>>> hf_new == eng_new, len(input_ids[0]) + len(eng_new)
(True, 20)
>>> tok.decode(eng_new)
'\xa0a\xa0young\xa0man who has been living in the city for'

```

比的是 token id 不是文本：解码后字符串相同只说明差不多，token 列表相等才排除”某一步选错但恰好解回同一个字符串”。断言写死在 `tests/lm_engine_test.py:40`。

必须切掉 prompt: `generate()` 的返回值包含 prompt token, `inference()` 只返回新生成的 (docstring 在 `inferenceLM/engine/lm_engine.py:76`)。忘了这步两边永远不等，而错误信息会把你引向”是不是 sampling 没关”。

$6 + 14 = 20$: `max_length` 管的是总长，由 `inferenceLM/engine/lm_engine.py:119` 强制，和 `generate()` 语义一致。

仓库里的正式版本是参数化的三个 case (`tests/lm_engine_test.py:16--40`)。这类差分测试的方法论——为什么拿 HF 当 oracle、oracle 本身错了怎么办——归 §5。

3.5 那句 `await asyncio.sleep(0)`

`inference()` 里有两行看起来可以删掉的代码 (`inferenceLM/engine/lm_engine.py:101` 和 `:107`)。”睡 0 秒”读起来像是没有作用。要讲清它，得先把两件都叫 `async` 的事分开。

第一层，GPU `async`: PyTorch 的 op 是 eager 执行 + 异步下发，`y = x @ x` 把 kernel 塞进队列就立刻返回一个 handle，CPU 继续往下跑；真正的等待发生在同步点，比如 `.item()`、`.cpu()`。第二层，Python `asyncio`: 单线程 event loop 上的 cooperative scheduling，一个 coroutine 能不能推进与 GPU 完全无关，只取决于有没有别的 coroutine 通过 `await` 把控制权还回来。retro §2.3 把这两层分开之后列了三条一旦混淆就会得出的错误结论：“用了 `asyncio` 所以 GPU/CPU 就 overlap 了”、“在 loop 里 `yield` 一下就能减少 GPU wait”、“`async inference` 本身就意味着 `compute parallelism`”。三条都不成立。

它做了什么。只有一件事：把控制权交还给 event loop 一次。`asyncio.sleep(0)` 被特殊处理——不注册定时器，就是一次裸 `yield`，让 event loop 跑一轮已经 ready 的回调然后立刻回来。落到这个系统里，“别的 coroutine”具体指 `inferenceLM/request_receiver/request_receiver.py:59` 那句 `await self.pending_queue.put(...)`。所以这两行买到的是：`decode` 每走一步，新 request 有一次进入 queue 的机会。retro §1.2 称之为 scheduling contract。

它做不到什么。不让 CPU 和 GPU 重叠。看顺序：`yield` 在 `prefill()` 返回之后、`.item()` 之前。event loop 拿到控制权的那一刻，GPU 的活刚下发完还没被同步；此时别的 coroutine 能跑，但只能跑到它们自己的下一个 `await`。控制权转回来后 `.item()` 照样从头 block 到 GPU 算完，这段时间 event loop 上谁也跑不了。整个 step 里其他 coroutine 拿到的是一个短窗口，不是一段与 GPU 并行的时间。

ADR-001 给这个状态起的名字是 **Weak Async** (`docs/decisions/adr001-inference.md:32`): “不是真正的 `async GPU execution`——CPU 在 `tensor.item()` 时仍会 block 等 GPU。真正的 GPU/CPU 并行需要 CUDA stream 或 event-based sync, defer 到 Week 3/4。”而这份 ADR 的 Status 至今仍是 **Proposed** (`docs/decisions/adr001-inference.md:3`)，同目录的 ADR-000 是 **Accepted**，整个 `docs/decisions/` 里只有这一份没升级。它诚实地反映了状态：这个 `async` 方案的作者自己没有把它定案。

还有一件必须说清的：今天所有测试都在 CPU 上跑，而 CPU 上没有异步 kernel 下发，`.item()` 不 block 任何东西。也就是说在当前测试环境里，这两句 `sleep(0)` 的全部效果就是一次纯粹的 `yield`，上面关于“GPU 还没算完”的讨论一行都还没被真实验证过。

Example 3.2 (`sleep_zero_race`): 用 `draft.py` 把 `event loop` 被占住这件事测出来

仓库根目录有一个 `draft.py`，就是为这件事写的草稿：一个每 50 ms 打时间戳的 `ticker`，加两个假的 inference 协程。

```

3  async def ticker():
4      for i in range(20):
5          print(f"tick {i} at {time.time():.2f}")
6          await asyncio.sleep(0.05)
7
8  async def fake_inference_v1():
9      """你的方案:sleep(0) + 直接 .item()"""
10     x = torch.randn(4096, 4096, device='mps')
11     for _ in range(10):
12         y = x @ x # enqueue heavy GPU work
13         await asyncio.sleep(0)
14         _ = y.sum().item() # block
15
16  async def fake_inference_v2():
17     """to_thread 方案"""
18     x = torch.randn(4096, 4096, device='mps')
19     for _ in range(10):
20         y = x @ x
21         _ = await asyncio.to_thread(y.sum().item())

```

`main()` 里 `v2` 那一行是注释掉的，所以这个实验要跑两次，中间手工改一行：

```

23  async def main():
24      await asyncio.gather(ticker(), fake_inference_v1())
25      # 看 ticker 在 v1 期间几乎完全停 tick
26
27      # await asyncio.gather(ticker(), fake_inference_v2())
28      # 看 ticker 在 v2 期间持续 tick

```

第一次直接跑；第二次把第 24 行注释掉、第 27 行取消注释，再跑一次。（注意它写的是 `device='mps'` 不是 CUDA——这是一台 Mac 上的草稿。没有 MPS 的机器要把两处改成 `'cpu'`，但那样两个版本几乎没有差别，因为 CPU 上不存在“kernel 已下发但没算完”这个状态。）

本机实测（Apple Silicon + MPS，各跑两遍，只记 tick 间隔；标称 0.05 s）：

```

# v1: sleep(0) + 直接 .item()
gaps: [0.15, 0.10, 0.05, 0.05, ... 0.05]    max 0.15    total 1.12 / 1.13
# v2: to_thread
gaps: [0.06, 0.05, 0.05, 0.05, ... 0.05]    max 0.06    total 0.98 / 0.97
# 标称: 19 个间隔 × 0.05 = 0.95

```

差别是真的，但比 `draft.py` 里那句注释（“看 ticker 在 `v1` 期间几乎完全停 tick”）小得多：只有开头两三个 tick 被拖长。原因是 10 次 `matmul` 在这台机器上总共只要 0.2 秒左右。把 `range(10)` 改成 `range(100)`，信号就没有歧义了：

```

# v1, range(100): gaps [0.14, 0.11, 0.10, 0.11, ... 0.11]    total 2.05
# v2, range(100): gaps [0.06, 0.05, 0.05, 0.05, ... 0.05]    total 0.97

```

v1 的每一个间隔都从 0.05 涨到 0.11——ticker 想每 50ms 醒一次，但每次醒来前都要排队等 `.item()` 把 event loop 还回来，整条 trace 被拖长一倍多。v2 全程纹丝不动。这个实验不能说明什么——这一段比上面的数字更重要：

- 它测的是 4096 见方的 MPS matmul，不是 GPT-2 的 forward。真实 decode step 的计算量小得多、kernel 数量多得多，阻塞时长量级完全不同，数字不能外推到 LMEngine。
- 它说明的是 event loop 被占住，不说明 GPU 利用率。两个版本下 GPU 做的 matmul 一样多、耗时一样；v2 没让 GPU 更忙，只是让另一个 coroutine 不再被拖。把它读成“v2 提升了吞吐”是错的。
- v2 为什么有效，依赖 PyTorch 在等待时释放 GIL 这一层行为。这一层本实验没有验证，只是我们相信的解释。
- `draft.py` 不在 `tests/` 里，没有断言、没有 CI，两次运行之间要手工改一行。它是 scratch，不是回归测试。每个数字都是单次测量，没有重复、没有方差、没有控制后台负载。

	fake_inference_v1	fake_inference_v2
让出方式	<code>await asyncio.sleep(0)</code> (裸 yield，不设定定时器)	<code>await asyncio.to_thread(...)</code> (真正挂起，直到线程返回)
让出位置	kernel 下发后、 <code>.item()</code> 前	<code>.item()</code> 整个搬进工作线程
ticker 是否继续	只在那一瞬； <code>.item()</code> 期间停	全程继续
实测 tick 间隔	0.15 / 0.10 后回落 (range(10)) 全程 0.10–0.11 (range(100))	0.05–0.06 全程 0.05 (range(100))
CPU/GPU 重叠	不产生	event loop 线程与等待重叠
代价	零：无线程、无额外同步	每步一次线程切换；隐含依赖 PyTorch 等待时释放 GIL；线程池大小成为新调优面
今天用的	是 (<code>lm_engine.py:101, :107</code>)	否

Table 4: `draft.py` 里两个假 inference 协程的对照。数字是本机 Apple Silicon + MPS 的单次实测，标称 tick 间隔 0.05s。注意“代价”一行：v2 不是免费的升级，它把一个隐含依赖引进了正确性论证里。

Checkpoint 3.5 (sleep_zero_semantics): `sleep(0)` 到底做了什么

不看代码回答：

- `await asyncio.sleep(0)` 和 `await asyncio.sleep(0.001)` 在 event loop 里的行为差别是什么？
- 在 `inference()` 里，这一句买到的好处是什么？指名道姓说出是哪个 coroutine 的哪一行因此有了机会。
- 把这两句删掉，`tests/` 里哪个测试会失败？

Hint 第三问要小心：先想清楚“行为变差”和“测试变红”是不是同一件事。

参考答案：(a) `sleep(0)` 是特例，不注册定时器，只做一次裸 yield：event loop 跑一轮已经 ready 的回调就回来。`sleep(0.001)` 会注册定时器，当前 coroutine 至少 1ms 之后才可能被恢复——在一个每 token 都走一次的循环里，那是白加了一毫秒的下限。

(b) 买到的是 `inferenceLM/request\_receiver/request\_receiver.py:59` 的 `await self.pending_queue.put(tokenized_data)` 有机会推进，也就是正在生成的 request 不会把新

request 的接收饿死。

(c) 一个都不会失败。今天没有任何测试断言 event loop 的公平性。tests/data_flow_test.py 里 test_inference_running_while_input_new_requests 看起来最接近，但它在两次提交之间 await asyncio.sleep(0.5) (tests/data_flow_test.py:74)，半秒足够整个 inference 跑完，所以就算 inference 独占 event loop 它也会绿。这正是 draft.py 存在的理由——它测的东西还没有变成断言。

代码位置: inferenceLM/engine/lm_engine.py:101、:107、docs/retro/week2-retro.md:57--104

Checkpoint 3.6 (weak_async_alternative): Weak Async 之外的三条路

不看代码回答：

- (a) "Weak" 弱在哪里？给一个可以被观测到的判据，不要形容词。
- (b) 举出两种能做到真正 CPU/GPU 重叠的做法，各说一个代价。
- (c) 这份 ADR 的 Status 是什么？和 ADR-000 一样吗？说明了什么？

Hint 第一问的判据可以是“存在一段时间，event loop 上没有任何 coroutine 能推进”。

参考答案: (a) 弱在让出发生在同步点之前，同步本身仍然独占线程。可观测判据：让另一个 coroutine 定时打点，若打点间隔明显大于设定值，就说明存在一段“event loop 上谁也跑不了”的时间——draft.py 测的正是这个（range(100) 时 0.05 变成 0.11）。

(b) 其一，把阻塞的同步搬到线程池（asyncio.to_thread，即 v2），代价是每步一次线程切换，且正确性依赖 PyTorch 在等待时释放 GIL。其二，CUDA event + 非阻塞轮询：下发后 record() 一个 event，再 while not event.query(): await asyncio.sleep(0)，代价是轮询烧 CPU 且需要真正管理 stream，复杂度高一个量级。ADR-001 把这两类统称为“CUDA stream 或 event-based sync”，defer 到 Week 3/4。

(c) ADR-001 是 Proposed (docs/decisions/adr001-inference.md:3)，ADR-000 是 Accepted (docs/decisions/adr000-request.md:3)；docs/decisions/ 下只有这一份停在 Proposed。说明这个决定被实现了、被依赖了、被写进了测试，但从没有被正式接受——代码先跑起来了，判断还悬着。

代码位置: docs/decisions/adr001-inference.md:3、:32

3.6 run() 是 launcher 不是 runner

```

61     def run(self) -> asyncio.Task:
62         """
63         Start the Inference Engine to keep fetching requests and run inference.
64         """
65         self.open = True
66         return asyncio.create_task(self._keep_get_request_and_inference())
67
68     def kill(self):
69         """
70         Shut down the Inference Engine and kill any active jobs.
71         """
72         self.open = False

```

run() 没有 async，返回一个 asyncio.Task。它不执行 engine loop，它注册 engine loop 然后立刻返回。retro §1.3 把这句话说成一条接口设计原则：“一个 function 的 return behavior，必须和

它在系统里的角色一致”——launcher 就该 non-blocking return, terminal execution 才该 block 到完成, 不能名字叫 run() 而语义混在中间。

反例是写成 `asyncdef run(self): await self._keep_get_request_and_inference()`。ADR-001 对它的分析在 `docs/decisions/adr001-inference.md:84`: `run()` 永远不返回, caller 拿不回控制权, 于是连 `kill()` 都调不到——因为 `kill()` 必须写在 `run()` 的下一行, 而那一行永远执行不到。这是一个自锁: 唯一能停下循环的开关, 被循环本身挡在了后面。落到测试里, 正确的形态是固定的三步 (launch、等完成信号、收尾), 在 `tests/` 里出现了六次: `inference_engine_test.py` 的:55、:90、:121, `data_flow_test.py` 的:35 和:65, `trained_model_integration_test.py` 的:123。

常见误解: `run()` 返回 Task 是为了以后能调用 `kill()`

ADR-001 论证 `run()` 必须返回 Task 时用的理由是“否则我们根本没有机会去调用 `kill()`”。所以这个设计的收益就是: caller 拿回控制权之后可以在合适的时机 `engine.kill()`, 让循环干净地退出。

论证成立, 但兑现方式和它说的不一样, 有两处对不上。第一, `kill()` 在整个仓库里没有任何调用点——`grep -rn "kill("` 只命中它自己的定义 (`inferenceLM/engine/inference\_engine.py:68`), 六处收尾用的全是 `task.cancel()`。所以真实收益不是“能调 `kill()`”, 是 caller 拿到了一个可以 `cancel` 的句柄, 那是 `asyncio.Task` 给的。第二, 就算调了也停不下来: `while self.open` 的条件只在处理完一条 request 之后才重新求值, 而循环体第一句是 `await self.pending_queue.get()`, 队列空时就阻塞在那里。所以 `kill()` 之后循环仍挂在 `get()` 上, 要等下一条 request 进来并被处理完才会退出——对一个“关掉引擎”的接口来说, 这语义是反的: 越空闲越关不掉。

这不是没被发现: `BACKLOG.md` 的 P0 第一条就是 engine lifecycle management, 要求明确的 shutdown 机制 (`cancel pending tasks + drain queue`), 并点名目前“依赖 `pytest event loop cleanup` 做兜底”。(那一条把协程名写成 `keep_fetching_requests`, 代码里实际叫 `_keep_get_request_and_inference`——又一处文档漂移。)

Checkpoint 3.7 (`run_returns_task`): 把 `run()` 改成 `async def`, 哪个测试会挂死

不看代码回答:

- 把 `run()` 改成 `async def` 并在里面 `await engine loop`, 但测试一个字不改 (还是 `task=inference_engine.run()`)。会发生什么? 在哪一行卡住?
- 如果同时把测试改成 `await inference_engine.run()`, 又会怎样?
- 这两种失败在 `pytest` 输出里长得一样吗? 哪一种更难查?

Hint 一个 `async def` 被调用但没有被 `await`, 函数体里的第一行执行了吗?

参考答案: (a) `run()` 返回一个协程对象, 函数体一行都没跑——`self.open` 还是 `False`, 也没有任何 task 被注册。于是没有消费者, `tests/inference\_engine\_test.py:56` 的 `await pending_queue.join()` 永远等不到 `task_done()`, 测试挂死在这一行; 后面的 `task.cancel()` 也会炸 (协程对象没有 `cancel()`), 但那行根本执行不到。同样的挂死会发生在 `inference_engine_test.py` 的:91 和:122、`data_flow_test.py` 的:36 和:77、`trained_model_integration_test.py` 的:124。

(b) 挂在 `await inference_engine.run()` 这一行。循环真的跑起来了, request 也会被处理完, 但循环永远不退出——它阻塞在 `inferenceLM/engine/inference\_engine.py:45` 等下一条。后面的 `join()` 和 `cancel()` 一行都执行不到。这正是 ADR-001 描述的自锁。

(c) 都是超时, 但 (a) 更难查: 症状在 `join()` 那一行, 病因在 `run()` 的定义里, 中间没有任何报错; Python 顶多在解释器退出时补一句 `RuntimeWarning: coroutine... was never awaited`,

很容易被 pytest 输出淹没。(b) 至少卡在你刚改的那一行上。

代码位置: `inferenceLM/engine/inference\_engine.py:61--66`、`tests/inference\_engine\_test.py:55--57`、`docs/decisions/adr001-inference.md:84`

3.7 try / finally 与 task_done()

```

43     while self.open:
44         # fetch request
45         tokenized_data = await self.pending_queue.get()
46
47         # Do inference & save back to buffer
48         self.request_store[tokenized_data.request_id].status = RequestStatus.
PROCESSING
49         try:
50             self.request_store[tokenized_data.request_id].generated_tokens = await
self.lm_engine.inference(
51                 tokenized_data,
52                 max_length=self.request_store[tokenized_data.request_id].
max_token_length
53             )
54             self.request_store[tokenized_data.request_id].status = RequestStatus.DONE
55         except Exception as e:
56             self.request_store[tokenized_data.request_id].status = RequestStatus.
FAILED
57             print(f"Request ID {tokenized_data.request_id} failed during inference
with error: {str(e)}")
58             finally:
59                 self.pending_queue.task_done()

```

`asyncio.Queue` 内部维护一个 `unfinished_tasks` 计数: `put()` 加一, `task_done()` 减一, `join()` 一直等它归零。这个计数和“队列里还有没有元素”是两件事——元素被 `get()` 走了计数并不减少, 必须显式 `task_done()`。

于是: 一条 `request` 中途抛异常而 `task_done()` 没被调用, `unfinished_tasks` 永远差一个, `join()` 永远不返回。而 `join()` 是所有测试等待“处理完了”的唯一手段。症状不是“这一条失败了”, 是整个测试套件挂死。BACKLOG.md 有一条 [BUG] 专门记着这件事 (BACKLOG.md:27--32), 理由写得很准。有意思的是这一条今天还挂在 P0 未完成区, 而代码 (`inferenceLM/engine/inference\_engine.py:58--59`) 已经实现了它——和 §2.2 那个枚举是同一类文档漂移, 只是方向相反: 那边文档超前于代码, 这边代码超前于文档。

测试是有的, 走了两条不同路径: `test_inference_engine_reject_prompt_equals_max_len` (`tests/inference\_engine\_test.py:70`) 走真实异常, `test_inference_engine_handle_lm_engine_exception` (`tests/inference\_engine\_test.py:99`) 用 `AsyncMock` 换掉 `LMEngine.inference`。两者都在异常之后照常 `await pending_queue.join()`——能返回, 就证明 `finally` 生效了。

代价: 这个 `except` 块吞掉了很多东西。 `except Exception` 之后只有两件事: 置 `FAILED`, 然后 `print` 一行 (`inferenceLM/engine/inference\_engine.py:57`)。代价是: `print` 不是 logger, 没有级别、时间戳和结构化字段; 没有 `traceback`, 只有 `str(e)`, 一个 `KeyError` 打出来是个裸 key, 看不出它来自 `inferenceLM/engine/inference\_engine.py:48` 还是模型内部; 异常类型全部拍平, `OOM` 和 `NotImplementedError` 都是同一个 `FAILED`; 错误信息没有写回 `request` (`RequestData` 上没有 `error` 字段, 见 §2 的表 2), 用户能看到的只有 `status=4` 和空的 `generated_tokens`; `FAILED` 之后没人再看它——没有重试、没有 `dead-letter`、没有告警。

顺带点出一处运气：task.cancel() 引发的 asyncio.CancelledError 在 Python 3.8 之后继承 BaseException 而不是 Exception，所以不会被这个 except 抓住，测试收尾时不会有 request 被误标成 FAILED。代码里没有注释说明这一点——它是对的，但不是被论证过的对。

Checkpoint 3.8 (task_done_finally): 为什么 task_done() 必须在 finally 里

不看代码回答：

- (a) 把 task_done() 挪到 try 块的最后一行（成功路径上），失败的 request 会导致什么？症状出现在哪个函数里？
- (b) queue.task_done() 和“队列里还有没有元素”是同一件事吗？
- (c) 这个 except 块除了置 FAILED，还应该做什么？列出三件今天没做的。

Hint 想清楚 awaitpending_queue.join() 到底在等一个什么值归零。

参考答案：(a) unfinished_tasks 永远差一个，join() 永不返回。症状出现在调用方——测试卡在 join() 那一行直到超时，而 engine 自己看起来一切正常（它已经去处理下一条了）。典型的“病因和症状不在同一个模块”。

(b) 不是。get() 只把元素取走，计数不变；只有显式 task_done() 才减一。所以队列可以是空的而 join() 仍然阻塞——§2 的 request_lifecycle_trace 里能直接看到：T2 时 qsize 已经是 0，但 join() 还没返回。

(c) 任选三件：用 logger 并打 traceback；把错误信息写回 RequestData（需要先加一个 error 字段）；按异常类型分流（OOM 可重试，NotImplementedError 不可重试，KeyError 是内部 bug 该告警）；加 metrics 计数；把失败的 request 送进 dead-letter 结构以便事后检查。

代码位置： inferenceLM/engine/inference_engine.py:49--59、tests/inference_engine_test.py:99--126、BACKLOG.md:27--32

3.8 分层：LMEngine 知道什么，不知道什么

retro §1.5 把这一周的分层结论写成两张清单：LMEngine 只做 pure compute，InferenceEngine 负责 orchestration。理由不是审美，是“一旦 LMEngine 开始知道 request lifecycle，coupling 会快速扩散”——而扩散的第一步通常极其自然：“inference 做完了顺手把 status 改成 DONE”，一行代码，看起来还省事。

LMEngine 知道	LMEngine 不知道
token id 列表 (lm_engine.py:95)	request_id (对象在手，从不读)
prompt 长度 (len(), :105)	RequestStatus (import 了，从不用)
max_length (形参, :73)	request_store 的存在
model 与 model.config (:89, :119)	pending_queue / task_done()
上一步的 token 与 KV (:106)	谁在等它、跑完之后谁写回
何时该停 (:113-119)	失败之后该怎么办

Table 5: LMEngine 的知识边界，逐条 grep 核对。右列第一行值得强调：inference() 收到的是完整的 TokenizedData，request_id 就挂在上头伸手可及，但代码里一次都没读过——grep -n "request_id" lm_engine.py 零命中。边界不是靠“拿不到”守住的，是靠“拿得到也不碰”守住的。

这条边界在行为上守住了：lm_engine.py 里没有出现过 request_id、request_store 或任何 queue，状态转移三处全在 inference_engine.py (:48, :54, :56)。但在声明上漏了——inferenceLM/engine/lm_engine.py:5--6 那两行 import 了 RequestData 和 RequestStatus，两个都从未被使用：

```
$ uv run ruff check inferenceLM/engine/lm_engine.py --select F401 --output-format=concise
lm_engine.py:5:38: F401 `inferenceLM.data.request.RequestData` imported but unused
lm_engine.py:6:45: F401 `inferenceLM.data.request_status.RequestStatus` imported but unused
Found 4 errors. # 另两条是 AutoTokenizer(:2) 和 typing.Dict(:4)
```

(同一条命令跑 `inference_engine.py` 是干净的。)这四行不影响任何行为，但它们让 `import` 清单说了假话：一个“不知道 `request status`”的模块，头上写着它 `import` 了 `RequestStatus`。实际的风险是，下一个来改这个文件的人（很可能是三个月后的自己）看到这行 `import`，会得出“这里本来就可以碰 `status`”的结论，然后顺手写下 `retro §1.5` 警告的那一行。`ruff` 已经是 dev dependency，只是还没有任何 CI 或 pre-commit 在跑它。

Checkpoint 3.9 (layer_ownership): status 更新该归哪一层

不看代码回答：

- (a) “inference 跑完把 `status` 改成 `DONE`” 这行写在 `LMEngine` 里会怎样？列出两个具体后果，不要只说“耦合变高”。
- (b) `LMEngine.inference()` 的入参里其实有 `request_id`。既然拿得到，说它“不知道 `request id`”是不是自欺欺人？
- (c) `lm_engine.py` `import` 了 `RequestStatus` 却从不使用。这只是一个 lint 警告，还是一个分层问题？

Hint 第一问从“这个模块以后还能不能被单独测试/复用”入手。

参考答案：(a) 其一，`LMEngine` 必须持有 `request_store` 才能改 `status`，于是 `__init__` 多一个参数，所有单测都要先造一个 `store`——今天 `tests/lm_engine_test.py:38` 只要 `LMEngine(model)` 一行。其二，§4 做 batching 时一次 forward 对应多条 `request`，“跑完改 `status`”会变成“跑完改一批 `status`”，而这批是谁由 `scheduler` 决定——compute layer 会被迫理解调度策略。`retro §1.5` 的“下周注意事项”就是这条：不要把 batch metadata 塞进 `LMEngine`。

(b) 不是自欺欺人，但要把话说准：边界不是“拿不到”，是“拿得到也不碰”，而且这一条可核实——`grep -n "request_id" inferenceLM/engine/lm_engine.py` 零命中。更强的版本是让 `inference()` 只收 `list[int]`，那样连拿都拿不到；今天没这么做，是因为 `TokenizedData` 还兼着“用 `__len__` 报 prompt 长度”的职责 (§2.1)。所以这里的边界确实靠纪律，不靠类型。

(c) 两者都是，后者更值得修。lint 警告是症状；真正的问题是 `import` 清单是一份公开的依赖声明，它现在声明了一条不存在的依赖。修法是删掉那两行——一秒钟的事，但需要有人在 CI 里跑 `ruff`，今天还没有。

代码位置：`inferenceLM/engine/lm_engine.py:5--6`、`inferenceLM/engine/inference_engine.py:48、:54、:56`、`docs/retro/week2-retro.md:233--290`

Downscaling Tip: 在 CPU 上跑完整的 engine 测试

这一章涉及的测试都不需要 GPU。模型是 `gpt2 (124M)`，`fixture` 是 `module` 级的，每个文件只加载一次权重：

```
uv run pytest tests/lm_engine_test.py -q --durations=5
uv run pytest tests/inference_engine_test.py tests/request_receiver_test.py -q
```

本机实测 (Apple Silicon, 纯 CPU 路径, 权重已缓存): 第一条 5 passed in 24.09s, 最慢的 `[long_seq]` 占 16.65s; 第二条 7 passed in 25.79s. `long_seq` 把 `max_length` 设成 1024

(`tests/lm_engine_test.py:20`), 跑到模型上下文上限, 而且 `generate()` 和我们的 `loop` 各跑一遍。想更快就把那个 1024 临时改小, 但不要删掉它——它是唯一会走到 `n_positions` 边界的 case。首次运行会下载 gpt2 权重 (约 500 MB)。

Resource requirements: CPU 即可, 无需 GPU/MPS; 两条命令合计约 50 秒 (权重已缓存)。
`draft.py` 的实验需要 MPS 或 CUDA 才有意义, 见 `sleep_zero_race`。

这一章的每个决定单独看都可以说“这样也行, 那样也行”。检验在 §4: 把一条 request 变成一批的时候, 只有 `loop` 体归自己所有、只有 `prefill` 和 `decode` 是两个能分别调度的单位、只有每个 `step` 之间有一个真实的让出点, `continuous batching` 才写得下去。这一章的代码是那一章的前提, 不是它的雏形。

4 从一次一条到 **continuous batching**：一次被自己推翻的设计

前两章里每一句话都能落到一行代码上。这一章不行：到 Week 3 为止，inferenceLM/ 下没有任何一个标识符叫 batch、scheduler 或 admission（唯一的三处匹配是 transformer_lm_adapter.py 注释里描述 tensor shape 的 (batch, seq)）。所以这不是“如何实现 continuous batching”的教程，而是一次设计复盘：把 2026-04-09 批准的 DESIGN.md 拿出来，对着 2026-04-21 的讨论记录逐条勾掉，看哪些结论还站得住，哪些已经死了但文档还没改。本章的代码位置多数指向 docs/ 下的文档而不是源码，这是刻意的——指向一个不存在的行号比不给行号更糟。

4.1 现行 loop 的结构性限制：症状不是根因

先看今天真正在跑的东西。整个消费循环只有十七行，§3 已经逐行讲过：第 45 行取一条请求，第 50 行把它整条跑完，第 59 行在 finally 里归还队列计数，然后回到第 43 行取下一条 (inferenceLM/engine/inference_engine.py:43)。一次循环迭代 = 一条请求从进入到结束的完整生命周期。那一节关心它怎么工作，这一节只关心它不能怎么工作。

一个很容易脱口而出的诊断是这样的：

常见误解：GPU 利用率低，所以要做 continuous batching。

一次只跑一条请求，decode 阶段每步只有 [1, 1] 这么一个 token 进 GPU，算力大部分时间闲着。把 batch 开大，GPU 利用率上去了，吞吐自然就上去了。

这句话在 Week 3 的设计讨论里被当场纠正过，纠正内容记在 docs/progress/week3-discussion.md 第 4 节的表格里，原话是：“GPU utilization 低” as root cause → *That's the symptom* → *Root cause: request lifecycle coupled to iteration lifecycle*。

区别不是修辞。如果你相信根因是利用率，你会去找让 GPU 更忙的办法：更大的 batch、更低的精度、算子融合。这些都可能有用，但它们都不能让第二条请求在第一条跑完之前拿到第一个 token。只要循环的一次迭代还等于一条请求的一生，第二条请求的排队时间下界就是第一条请求的全部 decode 步数，无论 GPU 有多空。这是一个结构约束，不是一个性能数字。

同一份文档的 §2.5 把这句话说得更完整：

Current loop: one iteration = one request's full lifecycle (fetch → prefill → decode loop → done → next request). This couples request lifecycle to iteration lifecycle, preventing any work interleaving.

于是解法的形状也就被定死了：不是“把 batch 调大”，而是把两个 lifecycle 拆开。新的循环里，一次迭代 = 所有 in-flight 请求各走一步 + 顺带接纳新请求；一条请求的一生横跨很多次迭代。

Checkpoint 4.1 (symptom_vs_root_cause): 现行 loop 的根因，不是 GPU 闲着

不看文档回答：

- 把 inference_engine.py 里的 while self.open: 循环理解成“一次迭代做一件什么事”？新 loop 里同一句话该怎么填？
- 如果我们只把 GPU 换成快十倍的卡，第二条请求的排队延迟会不会改善？为什么这说明利用率不是根因？
- 写出一个“利用率”完全解释不了、但“lifecycle 耦合”能解释的观察。

Hint 问自己：pending_queue.get() 这一行，在一条请求的一生中被执行几次？

参考答案: (a) 今天: 一次迭代 = 一条请求的完整生命周期 (fetch → prefill → 整个 decode loop → 写回 → 取下一条)。新 loop: 一次迭代 = 所有 in-flight 请求各前进一个 step, 外加一次 admission 决定。(b) 会按比例缩短, 但结构不变——排队的请求仍然必须等前一条整条生成完。把 GPU 换成无限快, 第二条请求的 TTFT 仍然等于第一条请求的全部 decode 时间。能被硬件消掉的是常数, 不是耦合。(c) 一条 max_token_length=200 的长请求后面跟一条 5 token 就结束的短请求: 短请求的 TTFT 被完全挡住。这跟 GPU 忙不忙无关, 是 head-of-line blocking。

代码位置: 症状与根因的对照见 docs/progress/week3-discussion.md:123 (§4 表格第 3 行), 完整表述在同文件 §2.5; 现行 loop 见 inferenceLM/engine/inference_engine.py:43。

4.2 continuous batching 的定义, 以及它对 scheduler 位置的裁决

Week 2 的 retro 里已经先一步把定义收敛好了。docs/retro/week2-retro.md §2.4 的原话是:

continuous batching 不是”并发地跑很多 request”, 而是”在 step granularity 上动态维护 batch membership”。

每个词都在排除一种误读。”不是并发地跑很多 request”排除掉”开 N 个 asyncio task 各跑一条”——那是 N 份互不相干的单请求循环, GPU 上依然一条一条排队, 只是队列从 Python 挪到了 CUDA stream。”step granularity”排除掉 batch 粒度: 不是攒够一批发车、跑完再攒下一批 (那是 static batching, DESIGN.md §4.2 已因 head-of-line blocking 否掉)。”动态维护 batch membership”是说 batch 是一个可变集合, 每个 step 边界上可以进入 (admission) 也可以出 (eviction)。

同一节还写下了这个定义最重要的推论:

scheduler 不是一个独立于 engine loop 的外置幻想层; 它应该嵌在 engine loop 的 step-by-step admission / execution policy 里。

这条推论直接判了一个设计的死刑: DESIGN.md §3 架构图里那个独立的 Scheduler 方块。如果 batch membership 是在 step 边界上维护的, ”选谁进 batch”就发生在两次 forward 之间, 它是循环体的一部分, 不是一个能被抽出来单独调用的组件。Week 2 retro”问题 1: 加不必要的中间层”讲的是同一件事。

Checkpoint 4.2 (continuous_batching_definition): 一句话定义 continuous batching

不看文档回答:

- (a) 用一句话给出 continuous batching 的定义。这句话里必须出现两个关键词, 是哪两个?
- (b) ”开 10 个 asyncio task, 每个 task 跑一条请求”——这是不是 continuous batching? 为什么?
- (c) 定义里的”动态”具体指哪两个操作?

Hint 定义不是关于并发数量的, 是关于集合的。

参考答案: (a) continuous batching 是在 step 粒度上动态维护 batch membership。两个关键词: step 粒度 (step granularity) 和 batch membership。少了前者会退化成 static batching, 少了后者会退化成”并发跑很多 request”。(b) 不是。那只是把串行排队从 Python 队列挪到了 GPU 上: 10 个 task 各自持有自己的 KV cache、各自发 [1, 1] 的 forward, 模型层面从来没有形成过一个 batch, membership 这个概念根本不存在。(c) 每个 step 边界上: admission

(把等待中的新请求加进 running batch) 与 eviction (把已经命中 stopping criteria 的请求移出 running batch, 并回收它那片 KV cache)。

代码位置: (尚无实现, 依据 docs/retro/week2-retro.md §2.4; 被否掉的 static batching 见 docs/designs/DESIGN.md §4.2。)

Checkpoint 4.3 (scheduler_placement): scheduler 应该放在哪一层

不看文档回答:

- (a) DESIGN.md 的架构图把 Scheduler 画成一个独立方块, 前面接 Request Receiver, 后面接 prefill queue 和 decode queue。这张图哪里错了?
- (b) 按 §2.4 的推论, scheduler 的正确形态是什么? 它的输入输出各是什么?
- (c) 为什么 Week 3 明确不做 pluggable scheduler interface?

Hint 先回答”选谁进 batch 这个决定发生在哪两件事之间”。

参考答案: (a) 它把一个 policy 画成了一个 layer。图上 scheduler 是数据流里的一站 (request 流过它、被它路由到某个 queue); 实际上 admission 决定发生在两次 forward 之间, 是循环体内的一个判断。(b) 一个 admission policy 函数: 输入是 pending 队列状态 + 当前 running batch 状态 (外加 KV cache 余量之类的资源约束), 输出是”这一轮 prefill microbatch 收哪几条”。它不持有请求, 也不拥有队列。(c) week3 讨论 §1 把它列进”What was NOT adopted”, 理由是 premature abstraction——第二种 policy 还不存在, 没有第二个实现就无从设计接口。

代码位置: (尚无实现, 依据 docs/retro/week2-retro.md §2.4 与 docs/progress/week3-discussion.md §1。)

4.3 backend 约束把设计逼回来: stage-homogeneous batching

定义收敛之后, 讨论里出现的第二个提案很自然: 既然一次迭代要让所有 in-flight 请求前进一步, 那把待 prefill 的 prompt 和正在 decode 的单 token 拼进同一个 forward, 用 <BOS> 分隔、配一个精心构造的 mask, 不就一次搞定了吗?

这个提案被 backend 挡了回来。docs/progress/week3-discussion.md §2.1 标了 CRITICAL:

HuggingFace model() cannot mix prefill and decode in one forward pass. HF attention_mask is [batch_size, seq_len] —a padding mask only.

关键在 attention_mask 的秩。它是二维的 [B, S], 语义是”第 b 条序列的第 s 个位置是不是真 token”。要把两条独立序列拼进同一行还互不可见, 需要表达的是”第 i 个位置能不能看见第 j 个位置”, 那是 per-pair 的 [S, S] (乃至 [B, 1, S, S]) 结构, 二维 padding mask 表达不了。真要做就得换成支持 block table 的自定义 attention kernel (FlashAttention、xformers 那一类), 而 GPT-2 的 HF 实现不接受 ragged/packed 输入。

被逼出来的结论就是 stage-homogeneous batching: 一个 batch 里所有序列必须处在同一个阶段。

”两次 forward”很容易被读成”两倍并行”, 实际上是”两倍串行”。文档写得很直白: *Sequential execution, not GPU-parallel. Two model() calls.* 一次迭代的墙钟时间是 prefill forward 加上 decode forward, 不是两者取最大值——这也是为什么 prefill-first 的代价必须被显式承认 (§4.5)。

这段约束是从 HF 的 API 形状推出来的, 而 API 会变。本仓库 pyproject.toml 写的是 transformers>=5.5.3, 实际装的是 5.5.3; 在这个版本上 outputs.past_key_values 已经不再是 lm_engine.py 类型注解里写的 List[Tuple[Tensor, Tensor]], 而是一个 DynamicCache 对象——

	Prefill microbatch	Decode microbatch
input_ids	[B_p, S_max]	[B_d, 1]
past_key_values	None	上一轮的 cache
attention_mask	[B_p, S_max] (padding)	[B_d, L+1] (padding)
一次迭代里的调用次序	第一次 model()	第二次 model()

Table 6: stage-homogeneous batching: 每个 engine iteration 两次独立的 model() 调用，顺序执行。这两次调用之间没有任何 GPU 并行——它们是 Python 里前后两条语句，第二次的 kernel 要等第一次的 kernel 排完队。依据 docs/progress/week3-discussion.md §2.1–§2.2。

不支持下标 (cache[0] 直接抛 TypeError)，from_legacy_cache / to_legacy_cache 在 5.x 里也已不存在。真要做 batching，这一层会是第一个绊脚的地方。

Downscaling Tip: 十行脚本验证 backend 约束，不需要 GPU

不要相信讲义（也不要相信设计文档）里写的 tensor shape，自己打一遍：

```
m = GPT2LMHeadModel.from_pretrained("gpt2"); m.eval()
ids = torch.zeros((2, 8), dtype=torch.long)
mask = torch.zeros((2, 8), dtype=torch.long)
mask[0, :5] = 1; mask[1, :8] = 1 # 第一条 prompt 只有 5 个真 token
out = m(input_ids=ids, attention_mask=mask, past_key_values=None, use_cache=True)
c = out.past_key_values
print(type(c).__name__, len(c.layers), tuple(c.layers[0].keys.shape))
# DynamicCache 12 (2, 12, 8, 64)
```

把 mask 换成三维再传进去，看它报什么错；这是理解“为什么不能 packed”最快的一条路。

Resource requirements: CPU 即可。权重已缓存时实测 1.6 秒；首次运行要先下载 gpt2 权重 (HF 缓存目录 525 MB)。

Checkpoint 4.4 (why_stage_homogeneous): 为什么一次迭代必须发两次 model()

不看文档回答：

- (a) HF 的 attention_mask 是什么形状、什么语义？它为什么表达不了 packed 序列？
- (b) “一次迭代两次 forward” 是不是意味着这两次可以并行？一次迭代的耗时怎么算？
- (c) 如果我们哪天真的想混合 prefill 和 decode，需要换掉哪一层？

Hint 想清楚 padding mask 是“逐位置”的，而 causal 隔离是“逐位置对”的。

参考答案: (a) [batch_size, seq_len]，二维，逐位置标记“是不是 padding”。把两条序列拼进同一行之后，需要禁止第二条的 token 看见第一条的 token，这是位置对之间的约束，需要 [S, S] 级别的 mask；二维 padding mask 没有表达这件事的秩。(b) 不能并行。它们是 Python 里前后两条语句，中间没有 stream 切换，文档明说 *Sequential execution, not GPU-parallel*。一次迭代耗时 $\approx$ prefill forward + decode forward。(c) attention kernel 本身：要能吃 block table / ragged 输入的实现 (FlashAttention、xformers 之类)，GPT-2 的 HF 实现不支持。这被 week3 讨论列为 Future (chunked prefill、paged KV cache 同一档)。

代码位置: (尚无实现，依据 docs/progress/week3-discussion.md §2.1–§2.2。今天单请求的两种 shape 见 inferenceLM/engine/lm_engine.py:36 与 inferenceLM/engine/lm_engine.py:

60。)

Example 4.1 (one_iteration_walkthrough): 一次迭代: 3 条 in-flight decode + 2 条 pending prefill

这是设计推演，Week 3 尚无代码。下面的循环体在仓库里不存在，标注了”（已实测）”的 shape 是拿 gpt2 在 CPU 上真跑出来的，其余部分是推论。

场景：running batch 里有 3 条请求正在 decode，它们的 KV cache 已经对齐到长度 13；pending queue 里有 2 条新请求，prompt 分别是 5 和 8 个 token；admission policy 决定这一轮把两条都收进来。

第一次 model(): prefill microbatch

```
input_ids      = [2, 8]          # 两条 prompt padding 到 S_max = 8
attention_mask = [2, 8]          # [[1]*5 + [0]*3, [1]*8]
past_key_values = None
# -> logits [2, 8, 50257]          (已实测)
# -> DynamicCache, 12 层, layers[i].keys [2, 12, 8, 64] (已实测)
```

第二次 model(): decode microbatch

```
input_ids      = [3, 1]          # 三条各自上一步生成的 token
attention_mask = [3, 14]         # 13 个历史位置 + 这一步
past_key_values = cache(seq=13)
# -> logits [3, 1, 50257]        (已实测)
# -> cache 增长到 seq=14, keys [3, 12, 14, 64] (已实测)
```

未验证的设计推演，三条

(1) 两片 cache 怎么并进一个 batch。下一轮要让 5 条一起 decode，就得把 [2, 12, 8, 64] 和 [3, 12, 14, 64] 并成 [5, 12, 14, 64]——seq 维不等长，得先把短的两条补到 14。DynamicCache 在 5.5.3 上没有任何 concat / merge 方法（已核对 [dir\(DynamicCache\)](#)），所以要自己写 torch.cat，还得保证 12 层 × (keys, values) 全对齐。这套代码不存在，也没被测试过。

(2) padding 补在哪一边。lm_engine.py 第 38 行取的是 outputs.logits[:, -1, :]。prefill batch 若右补零，那条 5 token 请求的最后一个位置是 pad，取出的 logits 就是错的；推论是必须左补，并让 attention_mask 屏蔽左侧 pad。这只是从 shape 推出来的，无测试覆盖。

(3) 请求结束后怎么从 batch 摘掉。这一步反而有现成工具：DynamicCache.batch_select_indices(indices) 沿 batch 维做子集选择，实测把 [3, 12, 14, 64] 选成 [2, 12, 14, 64]。但什么时候调用、和 request_store 的状态更新怎么排序，都还没有设计。

4.4 DESIGN.md 被推翻了什么

DESIGN.md 的日期是 2026-04-09，Status 至今写着 Approved。Week 3 的讨论发生在 12 天后，没有人回去改那份文档——§1.5 的文档时间线表已经把这层关系摆出来了，这里不重复，只做内容层面的逐项对账。下面这张表是本章最需要记住的东西：它把两份文档逐条对上，标出哪一行还能信。

撤回 (retracted): 每个 request slot 预分配 75.5 MB KV cache，12 GB 卡上可开约 146 个 slot

原文见 DESIGN.md §4.1 的资源预算表： $2 \times 12 \text{ layers} \times 12 \text{ heads} \times 1024 \text{ seq} \times 64 \text{ dim} \times 4 \text{ bytes} = 75.5 \text{ MB}$ ，可用 VRAM 按 $12 - 0.5 - 0.5 \approx 11 \text{ GB}$ 估，得到 $11 / 0.075 \approx 146 \text{ slots}$ ，并据此得出“4 条并发只需 302 MB，完全没有 memory pressure”。

条目	DESIGN.md v0 (2026-04-09)	Week 3 讨论 (2026-04-21) 后的状态	判定
KV cache 策略	§4.1 静态预分配：每 slot 按满 1024 context 分配 75.5 MB，12 GB 卡上约 146 slots，4 并发约 302 MB	新 loop 下没有重新给出 KV cache 策略；今天的 cache 由 HF 的 DynamicCache 动态增长、请求结束即丢弃，”预分配”没有执行主体。paged KV cache 被降级为 Future，触发条件是”移出 GPT-2 124M”	已取代
Batching	§4.2 以 request level 做 load in/off，完成即 evict，立刻补新请求；拒绝 static batching	完全一致，只是被表述得更准确：在 step 粒度上动态维护 batch membership	仍有效
调度规则	§4.3 两个 logical queue，每个 iteration 只跑 prefill 或只跑 decode；prefill 非空时按 2 prefill : 1 decode 交替	§2.2 锁定”一次迭代两次 forward，prefill 和 decode 都发生”；either/or 前提消失，2:1 的语义随之失效；admission policy 标为 OPEN	已取代
Iteration 的定义	§6 Q3：一个 iteration = 一次 forward pass，scheduler 决定跑哪个 queue	§2.2：一个 iteration = 一次 prefill forward + 一次 decode forward	已取代
项目定位	§1 一个多用户 CLI inference 平台（类 Ollama），§7 里程碑排到 Week 8 ”相对 naive baseline 50% speedup、60% memory reduction”	§1 改为 lab-grade research-serving runtime；范围收窄到 continuous batching、lifecycle management、data collection 三件事做对	已取代

Table 7: DESIGN.md 逐项对照。五行里四行已取代，唯一整条存活的是 §4.2 的 request-level batching——它恰好是当初写得最不像实现方案、最像原则的一条。表里藏着一个模式：死掉的都是数字和结构，活下来的是原则。75.5 MB、146 slots、2:1、50% speedup 这些能写进验收标准的东西一条没剩，而”完成即 evict，别让快请求等慢请求”活到了今天。

算术本身没错，失效的是它的执行主体。这套预算假设存在一个我们自己写的 KVCacheManager (§6 Q1: 用 torch.empty() 预分配、自己 track slot)，按满 context 切成定长 slot。今天没有这一层：cache 由 HF 在 forward 里自己 append 出来 (DynamicCache)，长度随生成增长，请求结束就整片扔掉。Week 3 讨论也没有恢复它，只把 paged KV cache 记成 Future。这段预算既不是当前行为，也不是已批准的下一步——它是一份悬空的规格。留着仍有用：真要写 cache manager 时，这张表的计算方法（尤其”按满 context 分配会浪费 60–80%”这个 tradeoff）还是正确的起点。

撤回 (retracted): prefill 非空时跑 2 轮 prefill 再跑 1 轮 decode (2:1 ratio)

原文是 DESIGN.md §4.3 的伪代码:

```
each iteration:
  if prefill_queue is non-empty:
    run 2 rounds prefill, then 1 round decode # 2:1 ratio
  else:
    run decode
```

这条规则的前提是同一节上一句话：每个 iteration 只跑 prefill batch 或 decode batch (V0 不混合)。在 either/or 模型下，“2:1”是一个有意义的量：它说的是时间片配额——每三次迭代里两次给 prefill、一次给 decode。

Week 3 §2.2 把迭代结构改成“每次迭代 prefill 和 decode 都跑（两次 forward）”之后，“轮次”这个计量单位没有了，2:1 失去了指称对象。文档的原话是：*Under both-in-one-iteration model, this ratio no longer directly applies.* 它不是被一个更好的比例取代了，而是整个 ratio 形式的调度语言不再适用，取而代之的问题变成了“每次迭代最多放多少条新请求进来” (admission policy)，而这个问题标记为 OPEN。

Checkpoint 4.5 (two_one_ratio_is_dead): 2:1 到底怎么死的

不看文档回答：

- (a) “2:1 prefill:decode” 这条规则依赖一个什么前提？这个前提是被什么推翻的？
- (b) 它是“被换成了别的比例”还是“整类规则失效了”？区别在哪？
- (c) 顶替它位置的那个问题叫什么？现在有答案吗？

Hint 先问“2:1 里那个 1 单位是什么”——是一次 forward？一次迭代？还是一个 token 预算？

参考答案：(a) 前提是 DESIGN.md §4.3 里的 either/or 模型：每个 iteration 只跑 prefill 或只跑 decode，所以“轮次”可以当配额来分。推翻它的是 §2.1 的 backend 约束链——不能混合 → stage-homogeneous → §2.2 锁定“每次迭代两次 forward，两个阶段都跑”。(b) 整类失效。既然每次迭代 prefill 和 decode 都发生，就不存在“两轮 prefill 配一轮 decode”这种可数对象了；换成 3:1 或 1:1 同样没有意义。这是语义失效，不是参数调整。(c) admission policy：每次迭代放多少条 pending 请求进 prefill microbatch。文档给了三个候选（全收 / 每轮最多 N 条 / 其它规则）并标记 OPEN，没有答案。

代码位置：（尚无实现。原规则见 docs/designs/DESIGN.md:111；失效判定见 docs/progress/week3-discussion.md:70，§4 表格第 4 行在 docs/progress/week3-discussion.md:124。）

4.5 现在什么是 LOCKED、什么是 OPEN、什么是 NON-GOAL

Week 3 讨论最有用的一点，是它对每条结论都标了状态。动手前先分清这三档，比记住任何一条具体结论都重要。

prefill-first 这条要特别小心：它是一个被明确承认了代价的选择，不是免费的优化。文档写的是 *Trade-off: decode ITL is worse — in-flight decode requests wait for prefill forward to complete*，随后跟着 *Acceptable for current lab workload (no streaming, short prompts)*。后半句是这条决定的有效期条款：工作负载一旦变成流式输出或长 prompt，这条 LOCKED 就该重新开。

Checkpoint 4.6 (prefill_first_tradeoff): prefill 优先，牺牲了谁

不看文档回答：

- (a) prefill 放在 decode 前面，优化的是哪个指标？牺牲的是哪个？
- (b) 这个牺牲在什么工作负载下会变得不可接受？
- (c) 如果把次序反过来 (decode-first)，哪个指标会变好、哪个会变差？

Hint 两个指标的英文缩写都在 §2.3 里，一个是 TTFT，一个是 ITL。

状态	内容	出处与理由
LOCKED	迭代结构：每次迭代 = 先一次 prefill microbatch forward（本轮被 admit 的新请求），再一次 decode microbatch forward（所有 in-flight 请求）；顺序执行，两次 model()	§2.2；被 §2.1 的 backend 约束逼出来的，不是偏好
LOCKED	阶段次序：prefill 在 decode 之前	§2.3；优化 TTFT。代价写在同一节：in-flight 请求的 ITL 变差
OPEN	admission policy：每轮收多少条 pending 请求。候选”全收”/”每轮最多 N 条”/”其它”	§2.4；必须在任务拆解里当成独立任务解决
NON-GOAL	pluggable scheduler interface；hooks / trace events / async writer；metrics 收集；token-budget scheduling；chunked prefill；schema 重设计；多硬件支持	§3 Explicit Non-Goals；理由分别是”没有第二个实现”、“Phase B”、“运行时先要正确”

Table 8: Week 3 的三档状态。依据 docs/progress/week3-discussion.md §2.2–§2.4 与 §3。

参考答案: (a) 优化 TTFT：新请求在到达的那一轮就能拿到第一个 token。牺牲 in-flight 请求的 ITL：它们每一步都要先等 prefill forward 跑完。(b) 适用条件是”no streaming, short prompts”，反过来就是失效条件：要流式吐字时 ITL 抖动直接被用户看见；prompt 变长时 prefill forward 变慢，每个 in-flight 请求每一步都要多等这段。(c) decode-first 让 ITL 更稳，但新请求的 TTFT 要多等一个 decode forward。lab 场景里”新请求马上有反应”更重要，所以选了 prefill-first。

代码位置：（尚无实现，依据 docs/progress/week3-discussion.md §2.3。）

Checkpoint 4.7 (admission_policy_open): OPEN 的那一条到底 open 在哪

不看文档回答：

- (a) admission policy 要回答的问题，用一句话说清楚。
- (b) 文档列了哪几个候选？”每轮全收”这个最简单的选项，会带来什么后果？
- (c) 为什么这条不能靠”先写着，跑起来再调”糊过去？

Hint 想想 prefill microbatch 的 $S_{\max}$ 是怎么定的，以及它和 KV cache 占用的关系。

参考答案: (a) 每次迭代最多把多少条 pending 请求收进这一轮的 prefill microbatch。(b) 三个：全收 / 每轮最多 N 条 / 其它规则。”全收”意味着 prefill microbatch 的 B_p 和 $S_{\max}$ 都不受控——突然来 20 条长 prompt，这一轮 prefill forward 很久，所有 in-flight 请求都被拖住（prefill 在前），KV cache 占用瞬间涨 20 片，没有任何背压。(c) 因为它决定的是最坏情况而不是平均性能。默认”全收”等于默认没有 admission control，之后再加要改的是 loop 结构和资源核算，不是一个常数。

代码位置：（尚无实现，依据 docs/progress/week3-discussion.md §2.4。）

到这里为止，这一章描述的都是一份自洽的设计。§1.5 在时间线表下留了一句话：week3-discussion.md 取代了 DESIGN.md，但它自己内部也有一处不一致，留到这一章展开。现在展开它——这条 checkpoint 要求你怀疑的不是自己的理解，而是文档本身。

Checkpoint 4.8 (ssot_contradicts_itself): 单一事实来源自己和自己打架

docs/progress/week3-discussion.md 是 CLAUDE.md 里指定的“batching/scheduling 的最新事实来源”（原话：supersedes DESIGN.md on batching/scheduling specifics）。它日期是 2026-04-21，全文 134 行，没有 Status 字段——没有任何一行告诉你它是提案还是定论。不看文档回答：

- (a) 这份文档的 §3 Goals 第 3 条写的是是什么？它和 §2.4 的结论是否一致？
- (b) 如果你照着 §3 的验收标准去写 Week 3 的代码，会写出什么？
- (c) 这是读者理解错了，还是文档的缺陷？该怎么修？

Hint 先把 §2.4 的标题和 §3 Goals 第 3 条并排抄下来，一个字一个字读。

参考答案：(a) 不一致，这是文档的一处自相矛盾。§3 Goals 第 3 条原文是 “2:1 prefill-prioritized scheduling (hardcoded in loop, no abstraction)”，位置在 docs/progress/week3-discussion.md:92。而同一份文档的 §2.4（标题就叫 *Scheduling Rule (OPEN — must be resolved in task breakdown)*）说 “Under both-in-one-iteration model, this ratio no longer directly applies”，并把 admission policy 列为待定；§4 的纠正表格第 4 行又写了一遍 “2:1 ratio still applies directly” 是错的，正确理解是 “Need new admission policy definition”。也就是说，同一份文档在 §2.4 和 §4 两处宣告 2:1 已死，却在 §3 的验收目标里把它原样留着。(b) 你会去 loop 里硬编一个 2:1 的计数器。但在“每次迭代两次 forward”的结构下，这个计数器没有可计数的对象——你只能自己发明一套语义（比如“每两轮迭代允许 admit 一次”），而那已经不是文档说的东西了。写出来的代码会和 §2.2 的 LOCKED 结构打架。(c) 是文档缺陷，不是读者理解错。读到互相矛盾的两节时，先怀疑文档。修法：把 §3 Goals 第 3 条改成“定义并实现一个 admission policy（候选见 §2.4），不做抽象接口”，让目标指向那个 OPEN 问题本身，而不是指向一个已经失效的比例。这正是 Week 2 retro” 问题 3：ADR 和代码不一致”的同一种病——只不过这次不一致的两边都在同一份文件里。

代码位置：（尚无实现。矛盾双方：docs/progress/week3-discussion.md:92 对 docs/progress/week3-discussion.md:68，佐证在 docs/progress/week3-discussion.md:124。）

4.6 给项目本身划界，也是一个设计决定

Week 3 讨论的第一节根本不谈 batching，谈的是这个项目要做什么。触发点是导师反馈：项目应该服务实验室的真实需求，而不是做一个 vLLM 的复制品。决定写得很直接：Project goal changed from “private vLLM clone” to “lab-grade research-serving runtime for local CoT models.” 理由可迁移：vLLM 的功能面（PagedAttention、CUDA/HIP graphs、chunked prefill、多硬件、OpenAI 兼容 server）是一个团队维护多年的产物，一个人八周只能在每一层上做到约 20%；而“研究型 serving runtime”把范围收成三件事——continuous batching、lifecycle management、data collection——做对且干净。文档给出的对比是 “designed and delivered a usable runtime for actual lab workloads” 强于 “attempted to replicate vLLM but only finished the first 20%”。这不是包装话术：它改变的是验收标准。前者问“某个真实工作负载能不能跑”，后者问“功能清单打了多少勾”；一个能被证伪，一个只能被稀释。

同一节还留下了一份很少见的记录——被明确拒绝的建议：

采纳	未采纳（及理由）
research-serving runtime 这个方向	报告给的 Week 3 时间表：8 个任务约 20 小时，over-scoped
分阶段 roadmap 结构（作为 backlog 参考）	pluggable scheduler interface：premature abstraction，没有第二个 policy
stage-homogeneous microbatch 方案	TraceEvent / hooks / async trace writer：属于 Phase B
lifecycle state machine 设计	schema 重设计（12 字段的 RequestRecord 等）：偏好增量演化
	metrics taxonomy：等 runtime 先能正确工作

Table 9: 一份外部建议报告的采纳与拒绝清单，出自 docs/progress/week3-discussion.md §1。右列比左列信息量大：它记下了“当时为什么没做”，而这正是几周后最容易被自己忘掉的东西。对 portfolio 项目来说，右列本身就是可展示的产物——只有左列的项目看起来像照单全收。

4.7 四条当场被纠正的说法

docs/progress/week3-discussion.md §4 是一张四行的表，标题 *Key Corrections Made During Discussion*，逐行记录“说了什么 / 错在哪 / 正确理解是什么”。第三行（GPU 利用率）已在 §4.1 用过，剩下三行原样放在这里。框里的内容是错的。

常见误解：每一轮判断是 **decode** 还是 **prefill**。

引擎循环每次醒来先做一个判断：这一轮是 prefill 轮还是 decode 轮？然后按判断结果跑对应的 batch。

错在把两个阶段当成互斥选项（either/or per iteration）。正确理解：一次迭代里两件事都发生，只是分成两次 forward。这个 either/or 的假设，正是 DESIGN.md 那条 2:1 规则赖以成立的地基。

常见误解：用 <BOS> 分隔符把 **prefill** 和 **decode** 打包进一个 **forward**，配一个 **mask** 就行。

把待 prefill 的 prompt 和正在 decode 的 token 拼成一条长序列，中间插 <BOS> 当分隔符，再构造一个 attention mask 让它们互相看不见，一次 forward 全部搞定。

错在低估了那个“配一个 mask”。它需要自定义 attention kernel：HF 的 model() 只吃 [B, S] 的 padding mask，不吃 per-pair causal mask (§4.3)。方向本身没错——vLLM 一类系统确实这么做——但代价是一个 kernel，不是几行 masking 代码。

常见误解：2:1 这个 **ratio** 还是直接适用的。

迭代结构虽然改了，但“prefill 优先、按 2:1 配比”这条调度规则可以照搬过来，只要在新 loop 里数着轮次执行就行。

错在 ratio 的语义随迭代模型一起变了。正确理解：需要一个新的 admission policy 定义。这条也是 §4.5 里那处文档自相矛盾的源头——纠正记在了 §4，却没有传播回 §3 的 Goals。

5 测试一个 async serving 系统

这个仓库的测试有一个很容易被忽略的特征：它几乎不 mock。tests/ 下九个文件、总共 880 行，unittest.mock 只出现在一个测试函数里；其余每一个测试都加载真的 gpt2 权重，跑真的 forward pass。

这不是懒。一个 inference engine 的核心断言是“我手写的 prefill/decode 循环和 HuggingFace 自己的 generate() 生成同一串 token”——把模型 mock 掉，这个断言就没有了对象。所以整套测试的主要手段是差分测试 (differential testing)：不描述正确输出长什么样，而是让被测实现和一个可信参照跑同一个输入，比对两者。

5.1 差分测试：让参照实现来定义“正确”

仓库里有两条差分链路，各自对着一个不同的参照。第一条是 Tokenizer 对拍 transformers. AutoTokenizer: tests/tokenizer_test.py 的八个用例全是这个形状——同一段文本喂给两边，断言 token 列表相等 (第 21 行)、decode 结果相等 (第 53 行)，空串、特殊字符、100 次重复的长文本各来一遍。第二条是 LMEngine.inference() 对拍 model.generate(): tests/lm_engine_test.py 第 25 行的 test_lm_engine_matches_huggingface 是全仓库最核心的一个测试——第 28–34 行调 HF 的 generate(), 第 35 行切掉 prompt 只留新 token, 第 38 行跑我们自己的循环, 第 40 行一句 assert hf_new_tokens == engine_new_tokens, 三组 max_length (30 / 1024 / 200) 走 parametrize。

这套办法有一个硬前提：两边必须是确定性的。只要采样带随机，逐 token 相等的断言就无从谈起。所以 do_sample=False 不是一个默认值，是差分测试的入场券。ADR-001 把这件事写进了决策理由 (adr001-inference.md “Inference Data Flow 的设计” 第 3 条)：

生成 new_token 的时候，用 greedy sampling，也就是选概率最高的 token 作为 new_token，因为这样是 deterministic 的，方便对比我们自己控制的 inference 和 model.generate() 是一致的。

顺便说一句这份 ADR 的状态：它的 Status 至今是 Proposed，不是 Accepted，尽管它描述的每一条决策都已经写进代码并被测试覆盖了。

第三条差分链路是后来加的，形状更精巧：tests/transformer_lm_adapter_test.py 第 15–31 行的 _LogitsOnly 把 GPT-2 剥成 (batch, seq) -> (batch, seq, vocab) 这个最朴素的签名——没有 config、没有 cache、没有输出 dataclass，正好长得像我们从零写的 TransformerLM——再拿它穿过 adapter 的无缓存重算路径去对拍 HF 带缓存的 generate() (第 62–83 行)。命题很干净：不带 KV cache 的贪心解码，必须和带 cache 的逐 token 相同；重算只会更慢，不会不一样。

Checkpoint 5.1 (differential_testing): 差分测试的前提和边界

不看代码回答：

- (a) 差分测试和“断言期望值”相比，各自适合测什么？这个仓库为什么主要用前者？
- (b) do_sample=False 在这里是性能选择还是正确性前提？说出理由。
- (c) 差分测试测不出哪一类 bug？举一个这个仓库里的具体例子。

Hint 想想“参照实现和被测实现共享了什么”。

参考答案：(a) 断言期望值适合输出短、可枚举、语义稳定的东西 (RequestData 的字段、异常类型)；差分测试适合输出长、且“正确”只能由另一个实现定义的东西 (一串 token)。这个仓库的核心产物正是后者——手写 prefill/decode 循环的输出没有别的方式能写出期望值。(b) 正确性前提。逐 token 相等的断言要求两边确定性；一旦 do_sample=True，两次运行取样不同，断言必然失败。ADR-001 明写选 greedy 就是为了能和 generate() 对比。(c) 测不出两边共有的错误。参照是 HF 自己，所以权重加载错了 (比如加载成另一个 checkpoint)

两边照样一致，测试全绿；tokenizer_test.py 八个用例都以 AutoTokenizer 为准，我们的封装和它一起错时测试也没有意见。

代码位置：tests/lm_engine_test.py:40、tests/tokenizer_test.py:21、tests/transformer_lm_adapter_test.py:83；理由见 docs/decisions/adr001-inference.md:52。

5.2 全仓库唯一一处 mock，以及它凭什么是合理的

unittest.mock 在这个仓库里出现四次，全部集中在同一个文件的同一个测试里：导入在 tests/inference_engine_test.py 第 10–11 行，使用在第 118 行和第 120 行。

```

118 runtime_error_inference = AsyncMock(side_effect=RuntimeError("Inference failed due to
    some error"))
119 # `patch` 需要完整的 import path
120 with patch(target = "inferenceLM.engine.inference_engine.LMEngine.inference", new=
    runtime_error_inference):
121     task = inference_engine.run()
122     await pending_queue.join() # Wait until the request in pending_queue is
    processed
123     task.cancel() # Cancel the background task after processing is done

```

被 mock 掉的是 LMEngine.inference，注入一个 RuntimeError；而这个测试要验证的东西在 LMEngine 之外：状态要变成 FAILED（第 125 行），generated_tokens 要保持为空（第 126 行），最要紧的是队列不能卡死——**finally** 块必须调到 task_done()（inferenceLM/engine/inference_engine.py:59）。规则于是可以写成一句话：mock 用来注入无法自然产生的失败，不用来替代被测对象。

对照同一文件里的另一个失败路径就更清楚：第 70 行的 test_inference_engine_reject_prompt_equals_max_len 同样在测 FAILED，但它没有 mock——构造一个 prompt 长度恰好等于 max_token_length 的请求，让 lm_engine.py 第 99 行那个真实的 RuntimeError 自己抛出来。能自然触发的失败就自然触发，只有“GPU OOM”这类在 CPU 上没法复现的故障才值得伪造。

Checkpoint 5.2 (when_to_mock): 什么时候才允许 mock

不看代码回答：

- (a) 仓库里唯一一处 mock 替换了什么？那个测试真正断言的是什么？
- (b) 同一个文件里还有一个测 FAILED 状态的用例却没 mock，两者的区别在哪？
- (c) 如果把 test_inference_engine_async_fetch_all_requests 里的模型也 mock 成“返回固定 token 列表”，会失去什么？

Hint 问“这个失败在真机上能不能自己发生”。

参考答案：(a) 替换 LMEngine.inference，注入 RuntimeError。断言的不是 LMEngine，而是 InferenceEngine 的错误处理契约：状态转 FAILED、不留半截 token、**finally** 里的 task_done() 照常执行所以队列不卡。(b) 区别是失败能不能自然产生。prompt 长度等于 max_length 是可构造的真实输入；而“inference 内部炸了”（OOM、driver 错误）在 CPU 上复现不了，只能注入。(c) 会失去它唯一的价值。它测的是十条请求经真实 async 循环走完并写回 request_store；模型一旦被替换，剩下的只是在验证“我们写的 mock 返回了我们写的值”。

代码位置：tests/inference_engine_test.py:118 与 tests/inference_engine_test.py:120；无 mock 的对照组在 tests/inference_engine_test.py:70。

5.3 async 测试的三行固定模式

每一个碰引擎的测试都长着同一个三行骨架，一个字都不差。它在仓库里出现六次（`inference_engine_test.py` 第 55、90、121 行；`data_flow_test.py` 第 35、65 行；`trained_model_integration_test.py` 第 123 行）。

Example 5.1 (`async_test_pattern`): 三行骨架，以及每行去掉会怎样

```
55 task = inference_engine.run()    # 不 await
56 await pending_queue.join()      # 等队列被消费干净
57 task.cancel()                  # 收尾
```

第 1 行：`run()` 不 `await`。它不是 `async def`，调 `asyncio.create_task()` 后立刻返回那个 `Task`（`inferenceLM/engine/inference\_engine.py:61`）。去掉会怎样：写成 `await inference_engine.run()` 就再也回不来——循环体是 `while self.open:`，只有 `kill()` 能把它置 `False`，而你已经没机会执行下一行去调它。测试永远挂住。这是 ADR-001 单列过的决定，§3 讲过。

第 2 行：`await pending_queue.join()`。它等的是未完成计数归零——每次 `put()` 加一，每次 `task_done()` 减一，而消费循环的 `finally` 块保证无论成功失败都会减（`inferenceLM/engine/inference\_engine.py:59`），所以归零等于“所有请求都处理完了（DONE 或 FAILED）”。去掉会怎样：下一行立刻 `cancel()`，引擎大概率还停在 `await self.pending_queue.get()` 上一条没跑完，后面断言全挂。

第 3 行：`task.cancel()`。往后台任务里扔一个 `CancelledError`，让它别再空转。去掉会怎样：测试返回了，那个 `task` 还挂在 `event loop` 上等下一条请求，留给 `pytest` 关闭 `loop` 时销毁（`asyncio` 通常会为此打一条告警）。它不会让当前测试失败，所以很容易被忽略——BACKLOG.md 的 P0 正是冲这件事去的。

第 2 行是这个模式里唯一一个需要想清楚的地方，因为有一个看起来更自然的写法：`await task`。

Checkpoint 5.3 (`queue_join_semantics`): 为什么是 `join()`，不是 `await task`

不看代码回答：

- (a) `pending_queue.join()` 到底在等什么？它和“引擎那个 `task` 结束”是不是一回事？
- (b) 把第 2 行换成 `await task` 会发生什么？
- (c) 是哪一行代码保证了 `join()` 一定能返回，哪怕请求推理失败？

Hint 先回答：`_keep_get_request_and_inference()` 这个协程，在正常情况下会不会返回？

参考答案：(a) 等未完成计数归零，即每个 `put()` 都配上了一次 `task_done()`。和 `task` 结束完全不是一回事：`task` 跑的是 `while self.open:` 长循环（`inferenceLM/engine/inference\_engine.py:43`），队列空了它就阻塞在 `await self.pending_queue.get()` 上继续等，永远不会自己结束。(b) 死锁。`await task` 等一个永不完成的 `task`；`kill()` 只把 `self.open` 置 `False`（`inferenceLM/engine/inference\_engine.py:72`），而循环这时正卡在 `get()` 里，连那个条件都读不到。测试一直挂到超时。(c) `finally: self.pending_queue.task_done()`（`inferenceLM/engine/inference\_engine.py:59`）。没有它，任何一条请求抛异常都会让计数永远差一，`join()` 永不返回——这正是 BACKLOG.md 那条 `try/finally` 条目要解决的。

代码位置：`tests/inference\_engine\_test.py:55-57`；契约在 `inferenceLM/engine/inference\_engine.py:43` 与第 59 行。

5.4 测试暴露出来的设计后果

测试写起来别扭的地方，通常是设计有话要说的地方。这里有两处。

其一：请求 ID 是全局单调的，测试必须先读一眼计数器。tests/request_receiver_test.py 第 16 行和第 54 行都有同一句 `request_idx=RequestReceiver.index`，然后才拿它拼出期望的 ID 列表。原因在 `inferenceLM/request\_receiver/request\_receiver.py:22`：index 是类属性，每次 `submit_request()` 自增（`inferenceLM/request\_receiver/request\_receiver.py:53`），新建实例不会清零；同一个 pytest 进程里 `data_flow_test.py` 每提交一条请求也在推它。这个文件第 5–6 行还把 `pending_queue` 和 `request_store` 写成模块级全局变量，三个测试函数共享同一份。

其二：有几个测试把实现细节钉进了字符串。tests/request_test.py 第 7–27 行逐字符断言 `str(RequestData)` 的输出，连换行位置和字段顺序都写死；同文件第 71 行断言一整条 JSON 字面量。RequestData 里任何一个字段改名、换位置、新增，这几个测试立刻红。要更正一个流传的说法：这不是全仓库唯一的脆性测试。同类断言至少五处——request_test.py 第 16–26、42–52、71 行三处黄金字面量，外加 `lm_engine_test.py` 第 51、67 行用 `pytest.raises(..., match=...)` 匹配完整错误消息（消息里还嵌着 `max_length` 的具体数值）。后两处 BACKLOG.md 已经记成 P2，前三处没人记。

Checkpoint 5.4 (test_isolation_leak): 测试之间漏了什么状态

不看代码回答：

- (a) 为什么 `request_receiver_test.py` 不能直接写 `request_ids=[str(i) for i in range(10)]`？
- (b) 除了 `RequestReceiver.index`，这个文件还共享了什么？
- (c) 这个泄漏是测试写法的问题，还是被测代码的设计问题？

Hint 找出 `index` 是在哪个作用域上定义的。

参考答案：(a) 因为 `index` 是类属性，在整个 pytest 进程里单调递增，跨函数跨文件都不重置。第一个跑的测试拿到 0–9，第二个从 10 开始，而“第一个”是谁取决于收集顺序，所以测试只能先读当前值再算期望。(b) 第 5–6 行的模块级 `pending_queue` 和 `request_store`，三个测试函数共用同一份。每个测试恰好提交 10 条又取走 10 条，队列才没有残留——这是巧合维持的平衡，不是隔离。(c) 是被测代码的设计问题，测试只是把它暴露出来。ID 分配是进程级全局计数器，既无注入点也无重置接口；将来跑多进程或需要可复现 ID 时就得改。

代码位置： `tests/request\_receiver\_test.py:16`、`tests/request\_receiver\_test.py:53`；根因在 `inferenceLM/request\_receiver/request\_receiver.py:22` 与 `inferenceLM/request\_receiver/request\_receiver.py:53`。

Checkpoint 5.5 (brittle_golden_string): 黄金字符串测的是契约还是格式

不看代码回答：

- (a) `test_request_data_str_matches_latest_format` 会因为哪些改动而失败？这些改动里有几个是真的破坏了契约？
- (b) 这类断言在什么情况下是正当的？
- (c) 同文件的序列化测试也断言了一整条 JSON 字面量，它和 `__str__` 那条是同一种脆性吗？

Hint 分开问两件事：这个字符串有没有下游消费者？它是不是接口的一部分？

参考答案: (a) 加字段、改字段名、调整 `__str__` 里的换行或顺序，全都会红，而其中只有“改名”算破坏契约。信噪比很低：它锁住的是 `inferenceLM/data/request.py:31` 那一行 f-string 的排版。(b) 当这个字符串有真实消费者时——日志格式、要被别的进程解析、或本身就是对外 API。现在没有：`__str__` 只在人眼调试时被读到。(c) 不完全一样。序列化那条断言的是 `model_dump_json()` 的输出，而序列化格式确实是跨进程契约（`adr000-request.md` 就是为它写的），钉死它的正当性强得多——它的失败是在提醒你“你改了对外格式”。

代码位置: `tests/request\_test.py:16` (`__str__` 黄金串)、`tests/request\_test.py:71` (JSON 字面量)；实现在 `inferenceLM/data/request.py:31`。

5.5 这套测试测不到什么

先把规模摆清楚。`uvrunpytest--collect-only-q` 的结果是 **44 tests collected**，来自八个文件：

文件	用例	覆盖什么	手段	真模型
<code>tokenizer_test.py</code>	8	encode/decode 与 HF tokenizer 一致；空串、特殊字符、长文本	差分	仅 tokenizer
<code>lm_engine_test.py</code>	5	手写循环对拍 <code>generate()</code> ；两条 <code>max_length</code> 越界拒绝	差分 + 异常断言	是
<code>inference_engine_test.py</code>	4	初始 <code>open=False</code> ；10 条请求全 DONE；两条 FAILED 路径	断言（含唯一 mock）	是
<code>data_flow_test.py</code>	2	Receiver → 队列 → Engine 全链路；边收边推理	断言	是
<code>request_test.py</code>	13	<code>RequestData</code> 的 <code>__str__</code> 、序列化与六种非法输入	黄金字面量 + 异常断言	否
<code>tokenize_data_test.py</code>	3	<code>TokenizedData</code> 的 <code>__str__</code> 与 <code>__len__</code>	断言	否
<code>transformer_lm_adapter_test.py</code>	6	adapter 无缓存重算对拍 <code>generate()</code> ；前缀截断与增长	差分 + 断言	是
<code>trained_model_integration_test.py</code>	0	自训练权重走完整请求路径	—	—
合计	44			

Table 10: `tests/` 下九个文件的覆盖面。最后一个文件贡献 0 个用例：它在第 36 行用 `pytest.importorskip("cs336_basics.lm")` 在模块层就跳过，再加第 38–41 行的环境变量 `skipif`，本机默认环境下根本不会被收集。八个文件里三种手段并存，但只有前两行是真正的差分测试。

下面是这 44 个用例测不到的东西。这份清单是照着代码一条条核出来的，不是印象。

1. 并发正确性。六处 `run()` 分布在六个测试里，每个测试只起一个引擎、配一个队列。多消费者抢同一个 `pending_queue` 的场景一次都没出现过，`request_store` 这个共享 dict 的并发写也从未被压过。今天没问题（只有一个 consumer），Week 3 之后会变成问题。
2. 内存。没有任何用例观察内存占用。“KV cache 用完即丢”（ADR-001 的 Consequences）没有测试为它背书。
3. 性能。`benchmarks/` 下只有一个 `.gitkeep`。延迟、吞吐、TTFT、ITL——这些词在 `DESIGN.md` 的验收标准里出现过，在测试里一次也没有。

4. 关闭路径。kill() 在 inferenceLM/engine/inference_engine.py:68, 但 tests/ 里没有任何一处调用它 (grep .kill(与 shutdown, 零命中)。所有测试都用 task.cancel() 绕过去。BACKLOG.md 的 P0 "Engine lifecycle management" 至今还开着。
5. 采样路径。do_sample=True 在 inferenceLM/engine/lm_engine.py:45 与 inferenceLM/engine/lm_engine.py:70 会抛 NotImplementedError, 而测试里它只作为 RequestData 的一个字段值出现过 (tests/request_test.py:38), 从没被送进 LMEngine。
6. Week 3 的全部内容。stage-homogeneous batching、admission policy、batch 内的 cache 拼接与驱逐——§4 讨论的每一件事, 既没有实现也没有测试。

第 4 条值得多说一句, 因为它是一个循环: 没有 shutdown() 所以测试只能 cancel(), 而 cancel() 能让测试通过, 所以没有人被迫去写 shutdown()。测试的绿色在这里不是证据, 是麻醉。

Downscaling Tip: 在没有 GPU 的机器上跑完整套测试

44 个用例全都能在 CPU 上跑: 每个文件用 module 作用域 fixture 加载一次 gpt2 (例如 lm_engine_test.py 第 7-13 行), 权重按文件加载而不是按用例。先 `uvrunpytest--collect-only-q` 确认收到 44 个, 再单跑 `uvrunpytesttests/lm_engine_test.py-v`。

Resource requirements: CPU 即可。首次运行会下载 gpt2 权重 (HF 缓存目录 525 MB)。lm_engine_test.py 里那个 max_length=1024 的用例会真的生成到上下文写满, 是整套里最慢的一个。

Epilogue

一份回顾式讲义如果只讲已经建好的部分，读完的感觉会比实际状况好太多。所以最后一节专门列缺口。下面每一条都当场核过——用 `ls`、`grep` 或者读文件本身——核的办法写在括号里，方便你几个月后再验一遍。

缺口清单

先做哪三个

排序不用重排——仓库根的 `ROADMAP.md` Part I 已经排好了，依据是依赖关系而不是价值大小。这里只把最前面三条抄过来，各配一句“第一步动什么”。

第一，**engine 生命周期状态机** (`INIT` → `RUNNING` → `DRAINING` → `STOPPED`)。它是唯一真正的阻塞项，也是 `BACKLOG.md` 里仅剩的有效 P0。今天 `kill()` 只把 `self.open` 置 `False`，而循环正卡在 `await pending_queue.get()` 上——空队列上的一个 killed engine 会永远挂着。第 4 章那条“测不到关闭路径”就是这件事的症状。第一步：给 `InferenceEngine` 加显式状态字段和 `asyncshutdown(drain=True)`，然后把测试里的 `task.cancel()` 一个一个换成它。换不掉的那个，就是设计还没想清楚的地方。

第二，把 **request lifecycle** 和 **iteration lifecycle** 拆开。这是 §4 全章的根因。第一步：把 `_keep_get_request_and_inference()` 从“取一条跑到底”改成“每轮迭代推进所有在飞请求各一步”，为此需要一个 in-flight 集合来存 (`request_id`, `latest_token`, `past_key_values`)。动手之前先解决一个文档冲突：`future-tasks.md` 分了 `Scheduler` 和 `Batcher` 两层，而 `week3-discussion.md` 明说不做 pluggable scheduler——先定哪份是现行意图，再写代码。

第三，**stage-homogeneous batching**。已经 `LOCKED`，理由是后端约束不是偏好 (§4.3)。第一步：每轮迭代改成两次 `model()` 调用，先 `prefill microbatch` 再 `decode microbatch`，并且从第一行代码起就记住这两次是顺序执行的。再往后是 admission policy——它在 `ROADMAP.md` 里排第四，并且是整条路线上唯一的一个还没有答案的设计问题；建议等前两步的循环真能跑起来、拿到真实的 TTFT/ITL 曲线之后再定，而不是反过来。

这个仓库到底是什么

数一下行数：`inferenceLM/` 下 13 个 Python 文件共 514 行（其中 4 个 `__init__.py` 是 0 行），`tests/` 共 880 行，`docs/` 下自己写的 markdown 共 1502 行（不含四份模板）。文档大约是源码的三倍，而测试比源码还多。

一个五百行的 Python 包，跑得动一条请求的完整 `prefill`→`decode`，支持一种采样策略 (`greedy`)，没有 HTTP 层，没有 batching，没有性能数据。按功能清单算，这份东西很薄。

但清单不是它的产物。真正留下来的是那些被否决的方案，以及每一次否决时写下的理由：为什么不用 `model.generate()` (`ADR-001`, `Status` 至今还是 `Proposed`)；为什么 `run()` 必须返回 `Task` 而不能 `await`；为什么不提前暴露 `do_sample`；为什么“GPU 利用率低”是症状不是根因；为什么 `DESIGN.md` 里那套 75.5 MB/slot 的预算表算得没错却已经失效；为什么 2:1 这个比例不是被换掉了而是整类规则失效了；以及最后那条——`week3-discussion.md` 的 §3 目标清单里，那条已经在 §2.4 和 §4 被宣告死亡的 2:1 规则，至今还原样留着。

功能可以在一个下午被别的库替代掉。上面这些判断不能。如果这份讲义只值得记住一件事，那就是：一个设计文档最有价值的部分，是它记下的那些“当时为什么没这么做”，以及它没能自我更新的那几行。

缺什么	怎么核实的
Week 3 一行代码都没有	<code>inferenceLM/</code> 下 <code>grep-rniE"batch schedule admission drain shutdown"</code> 只命中三处，全是 <code>transformer_lm_adapter.py</code> 注释里描述 <code>tensor shape</code> 的 (<code>batch, seq</code>)。git log 停在 Week3 start (2026-04-21)
<code>inferenceLM/output/</code> 是空的	目录里只有一个 0 字节的 <code>__init__.py</code> 。 <code>detokenize</code> 与流式回传在 <code>DESIGN.md §3</code> 的架构图里画着，从没落地。今天调用方只能轮询 <code>request_store[request_id]</code>
<code>benchmarks/</code> 是空的	目录里只有一个 <code>.gitkeep</code> 。这个仓库里没有任何性能测量代码；唯一一个真实穿过 <code>LMEngine</code> 的 <code>serving benchmark</code> 在姊妹项目里，不在这里
没有 HTTP 层	<code>grep-rniE"socket fastapi uvicorn http flask grpc"</code> 在 <code>inferenceLM/</code> 下零命中。系统的入口 API 就是 <code>RequestReceiver.submit_request()</code> 这个协程； <code>DESIGN.md §3</code> 画的“TCP/stdin”从未存在
没有 CI	没有 <code>.github/</code> 目录。 <code>ruff</code> 和 <code>mypy</code> 声明在 <code>pyproject.toml</code> 的 <code>dev</code> 依赖里（第 18–22 行），但全文件没有 <code>[tool.ruff]</code> 或 <code>[tool.mypy]</code> 段，也没有任何东西调用它们
<code>LICENSE</code> 是 0 字节	<code>ls -la</code> 显示 <code>size</code> 为 0。文件在，内容没有
<code>README.md</code> 还是脚手架	第 2 行标题写着 <code>[ProjectName---TOBEDECIDED]</code> ，第 7 行状态写着“Setup phase — project topic and tech stack under discussion”。全文 30 行，多数是目录树注释
两个状态值是死的	<code>RequestStatus.PREFILLING</code> 和 <code>DECODING</code> 定义在 <code>inferenceLM/data/request_status.py:5–inferenceLM/data/request_status.py:6</code> ，但 <code>grep</code> 全仓库，除了枚举定义本身和第 10 行那句注释之外，没有任何一处引用
<code>pytest-asyncio</code> 装错了位置	它在 <code>pyproject.toml:11</code> ，属于运行时 <code>dependencies</code> ，而不是 <code>[dependency-groups] dev</code> 。装这个包的人不一定要跑测试

Table 11: 核实过的缺口清单。前三条是能力缺失，后面几条是收尾没做完；两类的修复成本差着一个数量级，但都在同一份 `ROADMAP.md` 里排着队。